INSTITUTO FEDERAL DE GOIÁS

PÓS-GRADUAÇÃO EM INTELIGÊNCIA ARTIFICIAL

RELATÓRIO TÉCNICO E CIENTÍFICO

OTIMIZAÇÃO DE DIETAS E PESQUISA OPERACIONAL NO RuFaS

Disciplina: Modelagem e Otimização Aplicada — GYN.24551.2P.30

Professor: Dr. Eduardo Noronha

Alunos: Robson Pereira da Silva e Ivânio José da Rocha Júnior

Goiânia, Outubro de 2026

Resumo Executivo **3**
1. O Solver Linear e sua Implementação 4
1.1 Contexto Teórico da Programação Linear em Nutrição Animal 4
1.2 Formulação Matemática Canônica 5
1.3 Equações e Restrições Modeladas 5
1. Consumo de Matéria Seca Máximo (DMI Upper Bound) 5
2. Consumo de Matéria Seca Mínimo (DMI Lower Bound) 5
3. Exigência de Energia Líquida de Lactação 6
4. Exigência Mínima de Proteína Metabolizável (PM / MP) 6
5. Limite Superior de Proteína Metabolizável (Anti-Sobrecarga Hepática) 6
6. Exigência de Cálcio (Ca) 6
7. Exigência de Fósforo (P) 6
8. Fibra em Detergente Neutro Efetiva da Forragem (peNDF / fNDF) 6
9. Extrato Etéreo Máximo (EE / Gordura da Dieta) 6
1.4 Implementação Computacional no Código 7
Por que o algoritmo HiGHS? 8
1.5 O Solver Linear Nativo Autônomo: Método Simplex em Duas Fases do Zero 8
1.5.1 Conversão para a Forma Padrão Canônica 8
1.5.2 Estrutura do Método em Duas Fases 8
1.5.3 Regras de Decisão e Álgebra de Pivoteamento 9
1.5.4 Instrumentação para Inspeção Didática (Snapshots de Tableaus) 9
1.5.5 Explicação Passo a Passo do Código do LP Simplex (Arquitetura e Fluxo de Execução) 10
1. Inicialização e Parâmetros Numéricos (__init__) 11
2. Orquestração e Conversão Canônica (solve) 11
3. O Núcleo do Laço Simplex (_simplex_loop) 13
4. Operações Elementares de Gauss-Jordan (_pivot) 13
5. Conexão Zootécnica em ration_optimizer.py 14
2. Registro Completo das Modificações Realizadas 14
2.1 Modificações no Núcleo Biofísico (RUFAS/) 14
2.2 Modificações no Módulo Analítico e de PO (web_app/po_analytics.py) 15
2.3 Rastreamento e Gestão de Simulações (web_app/simulation_tracker.py) 16
2.4 Interface Web e Experiência do Usuário (web_app/static/) 17
2.5 Servidor e Rotas REST (web_app/app.py) 17
3. Explicações sobre a Interface Gráfica e seu Uso 17
3.1 Aba “Cenários & Fazendas” 18
3.2 Aba “Pesquisa Operacional & Insumos” 18
3.3 Aba “Painel Geral / Executar Simulação” 19
3.4 Aba “Histórico de Simulações” 20
3.5 Aba “ Tableaus Simplex” (Inspeção Didática Passo a Passo) 21
3.6 Aba “ Ficha das Vacas (IOFC)” (Desempenho Individual e Curvas de Lactação) 23
4. Limitações dos Modelos 26
4.1 Limitações do Modelo Linear (HiGHS Simplex) 26
4.2 Limitações do Modelo Não-Linear (SLSQP) 26
4.3 Limitações NASEM (2021) vs. NRC (2001) 27
5. Comparação Detalhada dos Resultados Encontrados 27

5.1 Matriz de Resultados Empíricos 28
5.2 Análise Comparativa dos Três Solvers 28
6. Estudo Experimental de Sensibilidade e Ajustes Paramétricos (output/sugestao_ajustes/) 30
6.1 Metodologia e Pipeline de Automação 30
6.2 Matriz Consolidada dos 10 Experimentos 31
6.3 Discussão e Interpretação dos Principais Achados 33
7. Conclusões e Recomendações 34
8. Guia Prático de Instalação, Configuração e Execução 34
8.1 Pré-requisitos de Sistema 34
8.2 Clonagem e Criação do Ambiente Virtual (venv) 35
No Windows (PowerShell ou Prompt de Comando): 35
No Linux / macOS (Bash / Zsh): 35
8.3 Instalação das Dependências 35
8.4 Modos de Execução da Aplicação 36
Modo 1: Interface Gráfica Web Interativa (Recomendado) 36
Modo 2: Execução em Linha de Comando (CLI / Headless) 36
Modo 3: Scripts de Verificação e Linting 37
8.5 Solução de Problemas Comuns (Troubleshooting) 37
Referências Bibliográficas 38

Projeto: RuFaS - Pesquisa Operacional & Simulação Biofísica
Ambiente: Python 3.13+, SciPy (`linprog` HiGHS / `minimize` SLSQP), Solver Simplex Nativo (Pure Python/NumPy), Flask Web App
Repositório: https://github.com/ivaniojr/RuFaS-PesquisaOperacional.git

Resumo Executivo

Este documento consolida o desenvolvimento, a implementação computacional, a validação agronômica e a integração sistêmica dos **Solvers de Pesquisa Operacional** no modelo biofísico **RuFaS** (*Ruminant Farm Systems*).

O ecossistema agora conta com **3 solvers integrados** para análise comparativa completa: 1. **Linear com Biblioteca (SciPy HiGHS):** Solver C++ de alto desempenho para simulações massivas de longo prazo. 2. **Linear Nativo Autônomo (Pure Simplex em Duas Fases):** Desenvolvido do zero em Python e NumPy sem bibliotecas de otimização de terceiros, com instrumentação analítica completa para inspeção dos **Tableaus Simplex** passo a passo. 3. **Não-Linear Mecanístico (SciPy SLSQP):** Solver gradiente com restrições não-lineares para ajuste fino e curvas biológicas contínuas.

O relatório está estruturado em 8 seções fundamentais:

1. **Os Solvers Lineares e suas Implementações:** Modelagem canônica de PL, arquitetura via SciPy HiGHS e formulação do Simplex Nativo em Duas Fases com álgebra de tableaus.

2. **Registro Completo das Modificações:** Alterações no núcleo do RuFaS, inclusão do módulo `pure_simplex_solver.py`, auditoria de falhas, supressão de métricas enganosas e rebalanceamento mineral.

3. **Guia de Operação da Interface Gráfica:** Descrição aprofundada dos módulos web, fluxos de uso, sensibilidade de preços, aba interativa **Tableaus Simplex**, aba "Ficha das Vacas (IOFC)" (desempenho individual e curvas de lactação) e controle de simulações.

4. **Limitações dos Modelos (Linear vs. Não-Linear / NASEM vs. NRC):** Pressupostos biológicos, trade-offs matemáticos e restrições de escala.

5. **Comparação de Resultados Reais:** Análise empírica comparando os três solvers com simulações de 365 dias, indicadores zootécnicos e econômicos (DMI, Custo, Leite, IOFC e Eficiência).

6. **Estudo Experimental de Sensibilidade e Ajustes Paramétricos:** Suíte automatizada de 10 experimentos (preços de insumos, tolerâncias, intervalo de reformulação, tamanho do rebanho, buffer de fósforo, preço do leite, aditivos de metano e padrão nutricional) executada nos três solvers, com análise da elasticidade bioeconômica da dieta.

7. **Conclusões e Recomendações:** Síntese zootécnica e metodológica para suporte à decisão.

8. **Guia Prático de Instalação, Configuração e Execução:** Requisitos de sistema, criação de virtualenv, instalação de pacotes e modos de execução (Interface Gráfica Web, Linha de Comando e Scripts Utilitários).

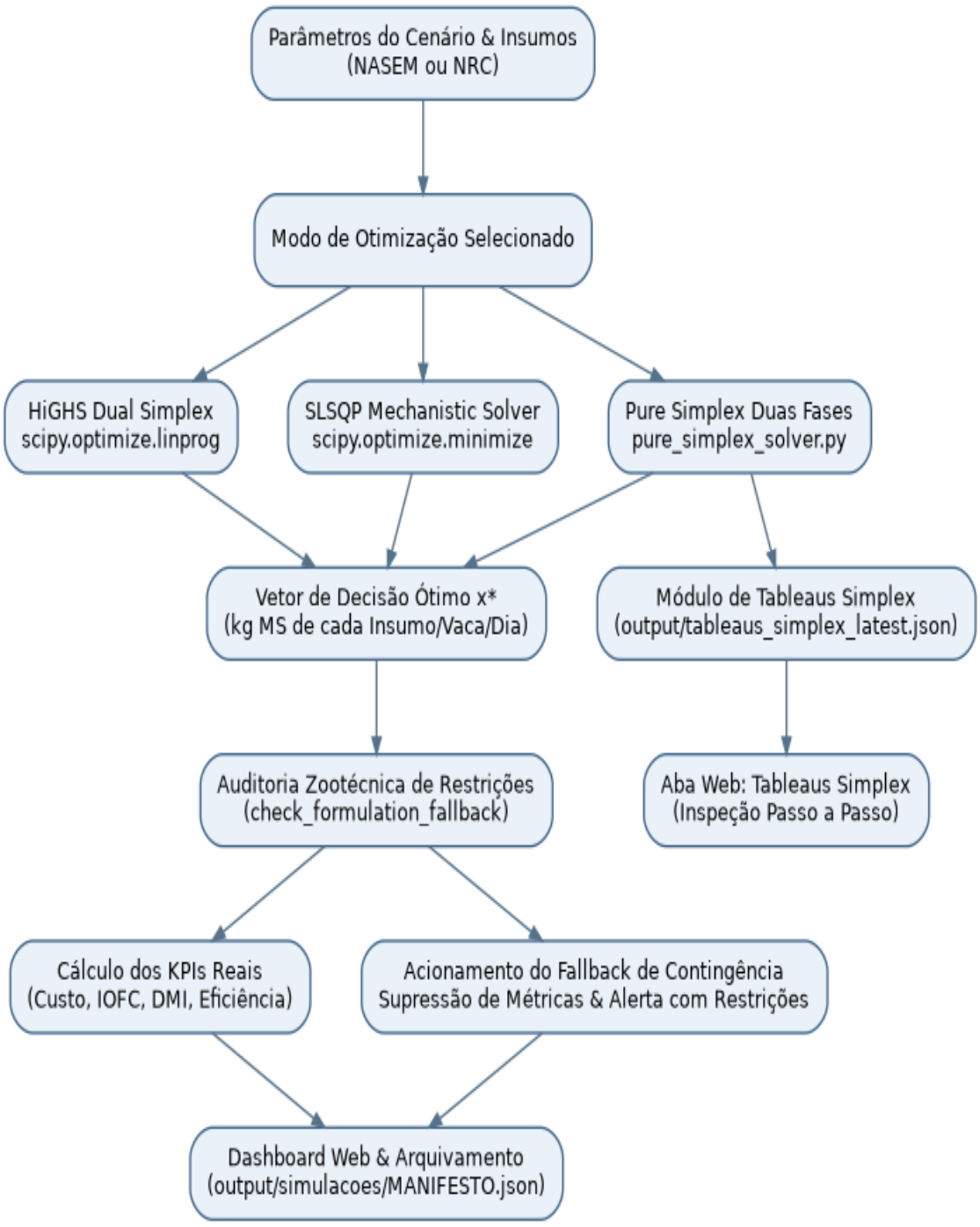


Figura 1 — Visão geral da arquitetura do sistema RuFaS.

---

1. O Solver Linear e sua Implementação

1.1 Contexto Teórico da Programação Linear em Nutrição Animal

O problema de formulação de dietas de mínimo custo (*Least-Cost Ration Formulation*) é historicamente um dos primeiros problemas clássicos resolvidos por Pesquisa

Operacional moderna, formalizado inicialmente por George Stigler (1945) e resolvido via método Simplex por George Dantzig (1947).

Em sistemas de produção de leite, a dieta alimentar representa entre **45% e 65% de todo o custo operacional efetivo** da fazenda. A formulação linear busca alocar uma cesta de n alimentos disponíveis no rebanho de forma a minimizar o desembolso financeiro diário, satisfazendo simultaneamente um conjunto de desigualdades lineares que representam as exigências fisiológicas, bromatológicas e digestivas dos animais.

1.2 Formulação Matemática Canônica

O problema formulado no RuFaS é definido no espaço $\mathbb{R}^n$ da seguinte forma:

$$\min_{\mathbf{x}} Z = \mathbf{c}^T\mathbf{x} = \sum_{i=1}^{n} c_i\, x_i$$

Sujeito a:

$$\mathbf{A}_{ub}\mathbf{x} \leq \mathbf{b}_{ub}$$

$$\mathbf{l} \leq \mathbf{x} \leq \mathbf{u}$$

Onde:

* $\mathbf{x} = [x_1, x_2, \ldots, x_n]^T \in \mathbb{R}^n$: Vetor de variáveis de decisão, representando a quantidade diária de cada insumo i ingerida por vaca, expressa em **quilogramas de Matéria Seca (kg MS/vaca/dia)**.

* $\mathbf{c} = [c_1, c_2, \ldots, c_n]^T \in \mathbb{R}^n$: Vetor de custos unitários de compra de cada insumo por quilograma de matéria seca ($/kg MS).

* $\mathbf{A}_{ub} \in \mathbb{R}^{m \times n}$: Matriz de coeficientes nutricionais dos insumos (bromatologia, energia, proteína, minerais e frações de fibra).

* $\mathbf{b}_{ub} \in \mathbb{R}^m$: Vetor de termos independentes, representando as demandas nutricionais e limites máximos recomendados para a categoria animal.

* $\mathbf{l}, \mathbf{u} \in \mathbb{R}^n$: Limites inferiores (*lower bounds*) e superiores (*upper bounds*) de inclusão de cada insumo.

1.3 Equações e Restrições Modeladas

O método linear no RuFaS traduz os requisitos nutricionais da vaca em 8 blocos de restrições de desigualdade linear ($\leq$):

1. Consumo de Matéria Seca Máximo (DMI Upper Bound)

A vaca possui capacidade física ruminal limitada para processar alimentos:

$$\sum_{i=1}^{n} x_i \leq \text{DMI}_{\text{req}} \cdot (1 + \delta)$$

onde δ é a fração de tolerância permitida pelo manejo (`dmi_constraint_fraction`).

2. Consumo de Matéria Seca Mínimo (DMI Lower Bound)

Para manter o fluxo ruminal e a saciedade produtiva:

$$\sum_{i=1}^{n} x_i \geq \text{DMI}_{\text{req}} \cdot (1-\delta) \Longrightarrow -\sum_{i=1}^{n} x_i \leq -\text{DMI}_{\text{req}} \cdot (1-\delta)$$

3. Exigência de Energia Líquida de Lactação (NE_L)

A energia fornecida pelos alimentos deve cobrir a mantença, a lactação e o ganho compensatório de peso:

$$-\sum_{i=1}^{n} \text{NEL}_i \cdot x_i \leq -NE_{\text{req}}$$

Implementação bromatológica: Se o insumo possui Energia Digestível (DE), calcula-se $\text{NEL}_i = DE_i \times 0.64$. Caso possua Nutrientes Digestíveis Totais (TDN), aplica-se a conversão estequiométrica $\text{NEL}_i = TDN_i \times 0.04409 \times 0.64$.

4. Exigência Mínima de Proteína Metabolizável (PM / MP)

Suprimento de aminoácidos absorvíveis no intestino delgado para síntese de caseína do leite:

$$-\sum_{i=1}^{n} \text{PM}_i \cdot x_i \leq -\text{PM}_{\text{req}}$$

onde $\text{PM}_i \approx CP_i \times 6.5$ g PM/kg MS.

5. Limite Superior de Proteína Metabolizável (Anti-Sobrecarga Hepática)

O excesso de proteína eleva os níveis de ureia plasmática, gera gasto energético para excreção no ciclo da ornitina e causa acidose metabólica:

$$\sum_{i=1}^{n} \text{PM}_i \cdot x_i \leq 1.15 \cdot \text{PM}_{\text{req}}$$

6. Exigência de Cálcio (Ca)

Mineralização óssea, contratilidade muscular e secreção no colostro/leite:

$$-\sum_{i=1}^{n} (\text{Ca}_i \times 10) \cdot x_i \leq -\text{Ca}_{\text{req}} \quad \text{(em gramas)}$$

7. Exigência de Fósforo (P)

Metabolismo de fosfocreatina, ATP, membranas celulares e integridade ruminal:

$$-\sum_{i=1}^{n} (\text{P}_i \times 10) \cdot x_i \leq -\text{P}_{\text{req}} \quad \text{(em gramas)}$$

8. Fibra em Detergente Neutro Efetiva da Forragem (peNDF / fNDF)

Estimulação da mastigação, salivação (tamponamento ruminal com bicarbonato) e prevenção de acidose ruminal subaguda (SARA):

$$-\sum_{i \in \text{Forragens}} (\text{FDN}_i - \text{FDN}_{\min}) \cdot x_i \leq 0$$

9. Extrato Etéreo Máximo (EE / Gordura da Dieta)

O excesso de lipídios livres (> 6% a 7% da MS) recobre as bactérias fibrolíticas do rúmen, colapsando a digestão de celulose e reduzindo o teor de gordura do leite:

$$\sum_{i=1}^{n} (\mathrm{EE}_i - \mathrm{EE}_{\max}) \cdot x_i \leq 0$$

1.4 Implementação Computacional no Código

A implementação do solver linear reside na classe `RationOptimizer`, no método `_attempt_linear_optimization`.

```python
# Trecho de RUFAS/biophysical/animal/ration/ration_optimizer.py
from scipy.optimize import linprog, OptimizeResult
import numpy as np

def _attempt_linear_optimization(
    self,
    ration_config: RationConfig,
    bounds: list[tuple[float, float]],
    animal_combination: AnimalCombination,
    initial_decision_vector: npt.NDArray[np.float64],
) -> OptimizeResult:
    # 1. Vetor de custos unitários c (purchase_cost por kg de MS)
    c = np.array(ration_config.price_list, dtype=np.float64)

    # 2. Construção da matriz A_ub e vetor b_ub das restrições
    A_ub_rows: list[list[float]] = []
    b_ub_rows: list[float] = []

    # Restrições de DMI, Energia, Proteína, Ca, P, FDN e Lipídios...
    # (vetorização estrita float64)
    A_ub = np.array(A_ub_rows, dtype=np.float64)
    b_ub = np.array(b_ub_rows, dtype=np.float64)

    # 3. Execução do Solver HiGHS Simplex
    res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=bounds,
method="highs")

    if not res.success:
        return OptimizeResult(
            x=res.x if res.x is not None else initial_decision_vector,
            success=False,
            status=res.status,
            message=f"Linear solver (HiGHS Simplex) failed:
{res.message}",
            fun=float(res.fun) if hasattr(res, "fun") and res.fun is
not None else 0.0,
            nit=int(res.nit) if hasattr(res, "nit") and res.nit is not
None else 0,
        )

    return OptimizeResult(
```

```
        x=res.x, success=res.success, status=res.status,
        message=res.message, fun=float(res.fun), nit=int(res.nit)
    )
```

Por que o algoritmo HiGHS?

- **Dual Simplex e Interior Point de Alto Desempenho:** HiGHS é a biblioteca C++ de estado da arte integrada ao SciPy a partir da versão 1.6, substituindo com ganhos massivos de precisão e velocidade os antigos métodos `'interior-point'` e `'simplex'` depreciados.
- **Complexidade Computacional:** Resolve instâncias de dietas leiteiras em **menos de 5 milissegundos**, comparado aos 80 a 300 milissegundos exigidos pelo SLSQP por iteração não-linear.
- **Garantia de Otimalidade Global:** Por tratar-se de um poliedro convexo $\mathcal{P} = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{A}_{ub}\mathbf{x} \leq \mathbf{b}_{ub}, \mathbf{l} \leq \mathbf{x} \leq \mathbf{u}\}$, o HiGHS garante matematicamente encontrar o vértice ótimo global (se a região viável for não-vazia), sem risco de prender-se em mínimos locais.

1.5 O Solver Linear Nativo Autônomo: Método Simplex em Duas Fases do Zero

Para fins acadêmicos, didáticos e de validação científica independente, foi desenvolvido e integrado um solver linear nativo autônomo em `RUFAS/biophysical/animal/ration/pure_simplex_solver.py`.

Este solver **não utiliza nenhuma biblioteca externa de otimização matemática** (como `scipy.optimize.linprog`, PuLP ou CVXPY), baseando-se exclusivamente em estruturas vetoriais e matriciais com NumPy para as operações elementares de linha de Gauss-Jordan.

1.5.1 Conversão para a Forma Padrão Canônica

O problema original com limites inferiores e superiores e desigualdades mistas ($\leq, \geq, =$):

$$\min Z = \sum_{j=1}^{n} c_j x_j \quad \text{s.a.} \quad \sum_{j=1}^{n} a_{ij} x_j \leq b_i \quad \text{e} \quad l_j \leq x_j \leq u_j$$

É convertido na forma padrão de igualdades:

1. **Normalização do Vetor b (RHS Não-Negativo):** Se qualquer $b_i < 0$, multiplica-se a linha inteira por -1, invertendo o sentido da desigualdade ($\leq$ transforma-se em $\geq$).

2. **Variáveis de Folga ($s_i \geq 0$):** Adicionadas às restrições do tipo $\leq$:

$$\sum_{j=1}^{n} a_{ij} x_j + s_i = b_i$$

3. **Variáveis de Excesso ($e_i \geq 0$) e Artificiais ($a_i \geq 0$):** Adicionadas às restrições do tipo $\geq$:

$$\sum_{j=1}^{n} a_{ij} x_j - e_i + a_i = b_i$$

4. **Tratamento dos Limites de Insumos ($l_j \leq x_j \leq u_j$):** * Limite inferior: $x_j \geq l_j \Longrightarrow x_j - e_k + a_k = l_j$ * Limite superior: $x_j \leq u_j \Longrightarrow x_j + s_k = u_j$

1.5.2 Estrutura do Método em Duas Fases

Como as restrições $\geq$ não admitem a origem ($x = 0$) como Solução Básica Viável inicial, adota-se o algoritmo de Duas Fases:

- **Fase 1 (Busca da Solução Básica Viável):**
  - Define-se uma função objetivo artificial auxiliar $W = \sum_{k=1}^{n_a} a_k$, cujo objetivo é minimizar o resíduo das variáveis artificiais.
  - O tableau inicial é canonicamente pivotado para zerar os custos reduzidos das variáveis artificiais básicas: $\text{Linha}_W \leftarrow \text{Linha}_W - \sum_{i \in \text{Base Artificial}} \text{Linha}_i$.
  - Se ao término da Fase 1 o valor ótimo $W^* = 0$, uma Solução Básica Viável real foi encontrada. Todas as colunas artificiais a_k são removidas da matriz e o sistema avança para a Fase 2.
  - Se $W^* > 0$, o sistema nutricional é **inviável** (não existe dieta física que atenda simultaneamente a todos os requisitos).
- **Fase 2 (Minimização do Custo da Ração):**
  - Restaura-se o vetor de custos reais da ração c_j.
  - Realiza-se o ajuste canônico da linha de custo Z:

$$\bar{c}_j = c_j - \mathbf{c}_B^T \mathbf{B}^{-1} \mathbf{A}_j$$

  - O algoritmo itera até que todos os custos reduzidos sejam não-negativos ($\bar{c}_j \geq 0$), indicando que nenhum insumo fora da base pode reduzir o custo diário da alimentação.

1.5.3 Regras de Decisão e Álgebra de Pivoteamento

A cada iteração do Simplex:

1. **Escolha da Variável que Entra na Base (Coluna Pivô k):**

$$k = \underset{j}{\text{argmin}} \{\bar{c}_j \mid \bar{c}_j < 0\}$$

Em caso de empate, utiliza-se a Regra de Bland (menor índice) para prevenção estrita de ciclos degenerados.

2. **Teste da Razão Mínima (Variável que Sai da Base - Linha Pivô p):**

$$p = \arg\min_{i \in \{1,\dots,m\}} \left\{ \frac{b_i}{a_{ik}} \;\middle|\; a_{ik} > \varepsilon \right\}, \quad (\varepsilon = 10^{-10})$$

3. **Atualização do Tableau (Gauss-Jordan):**

$$\text{Linha}_p \leftarrow \frac{\text{Linha}_p}{a_{pk}}$$

1.5.4 Instrumentação para Inspeção Didática (Snapshots de Tableaus)

O solver foi instrumentado com um coletor de telemetria que registra a matriz completa e o estado da base a cada passo da formulação:


```
{
  "step": 5,
  "phase": 1,
```

```
  "phase_name": "Fase 1: Viabilidade (min W)",
  "objective_value": 2984.3684,
  "entering_var": "Insumo_5",
  "leaving_var": "a_5",
  "pivot_row": 5,
  "pivot_col": 4,
  "pivot_val": 7.6863,
  "min_ratio": 2.2971,
  "basis_names": ["s_1", "a_1", "a_2", "a_3", "s_2", "Insumo_5", ...],
  "matrix": [[...]],
  "reduced_costs": [...]
}
```

Esse arquivo (`output/tableaus_simplex_latest.json`) alimenta diretamente o componente interativo da interface web, permitindo a navegação didática completa da resolução do modelo.

1.5.5 Explicação Passo a Passo do Código do LP Simplex (Arquitetura e Fluxo de Execução)

O módulo `pure_simplex_solver.py` implementa a classe `TwoPhaseSimplexSolver`, projetada para resolver com exatidão analítica o problema de programação linear canônico da ração bovina, sem recorrer a bibliotecas externas de otimização (como SciPy, GLPK ou COIN-OR), utilizando exclusivamente álgebra linear com NumPy.

Abaixo é apresentado o detalhamento do funcionamento de cada bloco lógico, estrutura de dados e método do algoritmo:

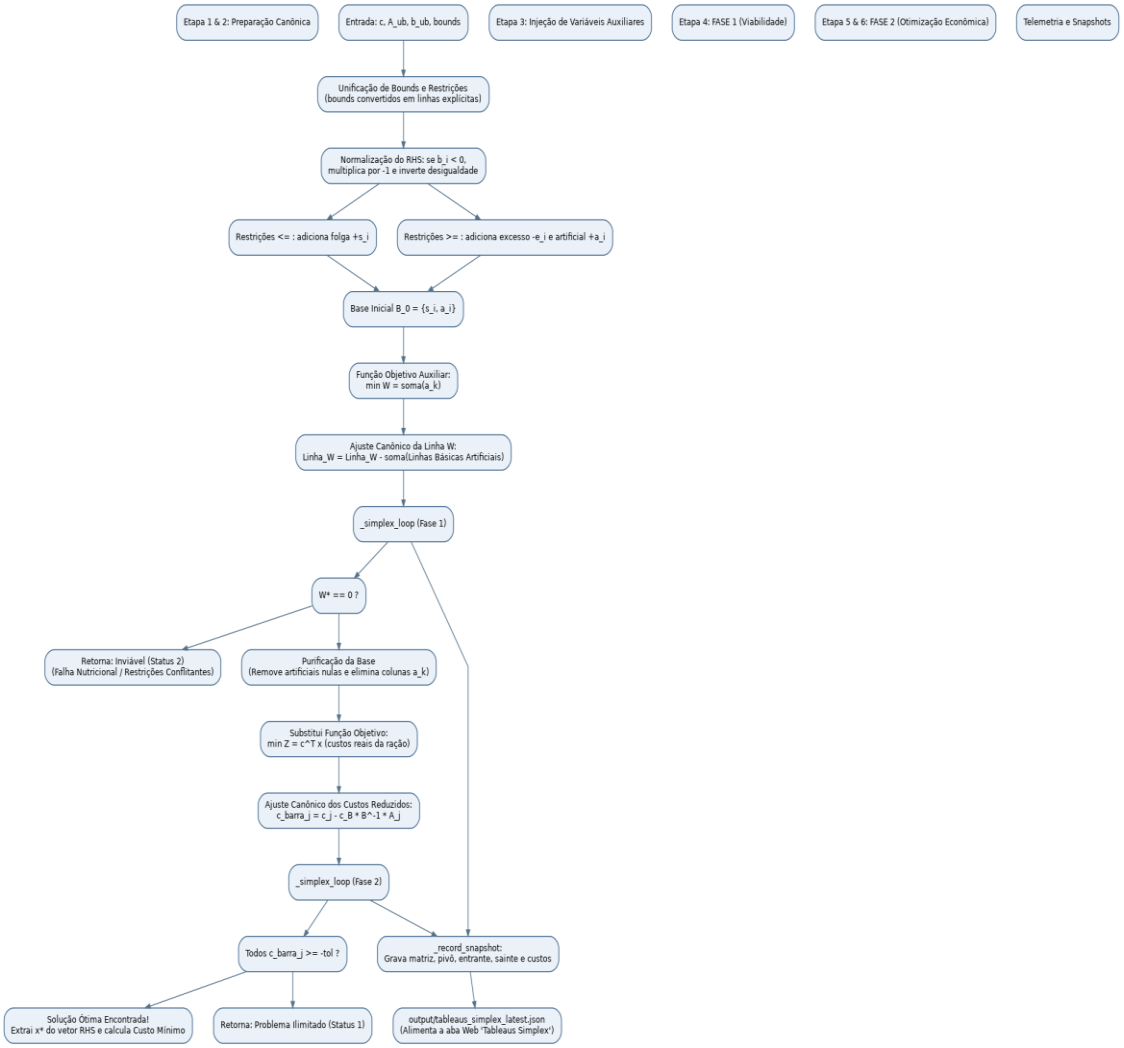


Figura 2 — Funcionamento do Simplex.

1. Inicialização e Parâmetros Numéricos (`__init__`)

- **`tol = 1e-8`:** Tolerância numérica escalar para validação de nulos em aritmética de ponto flutuante IEEE 754 de precisão dupla (`float64`). É utilizada para evitar instabilidades numéricas, como divisões por quase-zero e falsos coeficientes negativos causados por resíduos infinitesimais de máquina.
- **`max_iter = 2000`:** Teto máximo de pivoteamentos permitidos, prevenindo laços infinitos em problemas patológicos ou em situações de degenerescência com ciclagem.
- **`history`:** Lista sequencial em memória que acumula os metadados e os snapshots dos tableaus gerados ao longo de todas as iterações da resolução.

2. Orquestração e Conversão Canônica (`solve`)

O método `solve(c, A_ub, b_ub, bounds, var_names, constraint_names)` executa a conversão estrutural completa do problema em 6 passos:

1. **Unificação de Restrições e Limites:** * Recebe o vetor de custos $\mathbf{c} \in \mathbb{R}^n$, as restrições nutricionais $\mathbf{A}_{ub}\mathbf{x} \leq \mathbf{b}_{ub}$ e os intervalos de tolerância dos alimentos $[l_j, u_j]$. * Os bounds de inclusão são transformados em restrições matriciais convencionais: * Limite inferior: $x_j \geq l_j \Longrightarrow x_j \geq l_j$ (restrição do tipo $\geq$). * Limite superior: $x_j \leq u_j \Longrightarrow x_j \leq u_j$ (restrição do tipo $\leq$).

2. **Normalização do Termo Independente ($b_i \geq 0$):** * O algoritmo Simplex exige que o vetor do lado direito ($\mathbf{b}$) seja estritamente não-negativo para garantir que a base inicial seja geometricamente viável no primeiro quadrante. * Se algum $b_i < 0$, a linha inteira da matriz é multiplicada por -1, invertendo o sentido da desigualdade ($\leq$ converte-se em $\geq$, e vice-versa).

3. **Injeção de Variáveis Auxiliares (Folga, Excesso e Artificial):** * Para cada restrição do tipo $\leq$, adiciona-se uma coluna de folga $+s_i$ e define-se s_i como variável básica inicial. * Para cada restrição do tipo $\geq$, subtrai-se uma variável de excesso $-e_i$ e adiciona-se uma variável artificial $+a_i$. A variável artificial a_i é designada como variável básica inicial, garantindo que o bloco inicial das variáveis básicas forme a matriz identidade $\mathbf{I}_m$.

4. **Execução da Fase 1 (Eliminação das Variáveis Artificiais):** * Monta-se a matriz estendida inicial da Fase 1: $\begin{bmatrix} \mathbf{A}_{\text{std}} & \mathbf{b}_{\text{std}} \\ \mathbf{c}_{p1}^T & 0 \end{bmatrix}$, onde o vetor de custos artificial inicial possui coeficiente 1 para variáveis artificiais e 0 para as demais. * **Ajuste Canônico Inicial:** Para que o tableau esteja na forma canônica com respeito à base inicial, zera-se o custo reduzido das variáveis artificiais básicas subtraindo suas respectivas linhas da linha da função objetivo:

* Invoca-se `_simplex_loop(tableau_p1, ...)`. * **Diagnóstico de Viabilidade:** Ao concluir a Fase 1, avalia-se o valor ótimo da função artificial $W^* = -\text{tableau}[-1,-1]$. Se $W^* > 10^{-4}$, resta resíduo artificial que não pôde ser eliminado; logo, a região viável é vazia ($\mathcal{P} = \emptyset$) e o sistema retorna imediatamente indicando inviabilidade agronômica. * **Purificação da Base:** Caso alguma variável artificial permaneça na base com valor numérico zero (caso degenerado), ela é forçada para fora da base através de um pivoteamento com qualquer coluna não-artificial com coeficiente não-nulo. * **Remoção de Colunas:** Todas as colunas correspondentes às variáveis artificiais a_k são removidas da matriz NumPy, reduzindo a dimensionalidade para a Fase 2.

5. **Execução da Fase 2 (Minimização Econômica da Dieta):** * Instancia-se a matriz estendida da Fase 2 contendo as restrições purificadas e o vetor de custos reais da ração: $\mathbf{c}_{p2} = [c_1, \ldots, c_n, 0, \ldots, 0]$. * **Ajuste Canônico dos Custos Reduzidos:** A linha de custos reais é canonicamente projetada na base atual através da subtração:

$$\text{Linha}_{m+1} \leftarrow \mathbf{c}_{p2} - \sum_{i=1}^{m} c_{B(i)} \cdot \text{Linha}_i$$

* Invoca-se `_simplex_loop(final_tableau, ...)` para percorrer as arestas do poliedro convexo em direção ao vértice de mínimo custo financeiro diário.

6. **Extração da Solução Ótima:** * A solução é lida diretamente da coluna de termos independentes (RHS, última coluna do tableau): para cada linha i, se a variável básica $\text{basis}[i]$ for uma das variáveis de decisão originais ($< n$), define-se $x_j^* = \text{tableau}[i,-1]$; as variáveis não-básicas assumem valor 0. * Calcula-se o custo total ótimo $Z^* = \mathbf{c}^T\mathbf{x}^*$.

3. O Núcleo do Laço Simplex (`_simplex_loop`)

O método `_simplex_loop` implementa o ciclo clássico de transição entre vértices adjacentes da Programação Linear:

1. **Critério de Otimalidade de Dantzig (Escolha da Coluna Pivô):** * Avalia o vetor de custos reduzidos na última linha do tableau: $\bar{\mathbf{c}} = \text{tableau}[-1, :q]$. * Localiza o índice do menor custo reduzido: $k = \text{argmin}_j \bar{c}_j$. * Se $\bar{c}_k \geq -\text{tol}$, não existe nenhum insumo que possa reduzir o valor da função objetivo. A base atual é comprovadamente **ótima**, e o laço encerra com sucesso. * Caso contrário, a variável associada à coluna k é eleita como a **variável entrante na base**.

2. **Teste da Razão Mínima (Escolha da Linha Pivô):** * Para garantir a não-negatividade das variáveis ($x \geq 0$), calcula-se a razão entre os termos independentes atuais e os coeficientes estritamente positivos da coluna entrante:

* Se nenhum coeficiente for estritamente positivo ($a_{ik} \leq \text{tol}, \forall i$), a variável pode crescer indefinidamente sem violar nenhuma restrição. O problema é matematicamente **ilimitado** (*unbounded*), e o laço encerra com erro. * A linha que atinge a menor razão positiva ($p = \text{argmin}_i \theta_i$) determina a restrição mais restritiva. A variável básica atual da linha p é a **variável sainte da base**.

3. **Registro de Telemetria (`_record_snapshot`):** * Antes de alterar os valores da matriz, salva um registro completo contendo: iteração atual, fase (1 ou 2), nomes das variáveis de entrada e saída, elemento pivô a_{pk}, vetor de razões θ, matriz com coeficientes arredondados e valor corrente da função objetivo.

4. **Pivoteamento:** Invoca `_pivot(tableau, basis, p, k)`.

4. Operações Elementares de Gauss-Jordan (`_pivot`)

O método `_pivot(tableau, basis, pivot_row, pivot_col)` atualiza o tableau matricial in-place no NumPy:

1. **Normalização da Linha Pivô:** * Divide todos os elementos da linha p pelo elemento pivô escalar a_{pk}:

* Após essa divisão, o elemento na posição (p, k) torna-se identicamente 1.0.

2. **Eliminação Gaussiana das Demais Linhas: * Para cada linha $i \in \{0, \ldots, m\}$ com $i \neq p$**: * Obtém o fator de cancelamento $f = \text{tableau}[i, k]$. * Se $|f| > 10^{-15}$, subtrai o múltiplo da linha pivô normalizada:

$$\text{tableau}[i, :] \leftarrow \text{tableau}[i, :] - f \cdot \text{tableau}[p, :]$$

* Essa operação zera todos os coeficientes da coluna k, exceto o pivô, convertendo a coluna entrante em um vetor unitário canônico $\mathbf{e}_p$.

3. **Atualização do Vetor de Base:** * Define $\text{basis}[p] \leftarrow k$, indicando formalmente a entrada da nova variável na base e a saída da anterior.

5. Conexão Zootécnica em `ration_optimizer.py`

A ponte entre o modelo biológico do RuFaS e o solver matricial ocorre no método `_attempt_native_simplex_optimization`:

* **Mapeamento de Restrições:** Traduz as exigências nutricionais das vacas calculadas pelo motor NASEM/NRC em linhas da matriz $\mathbf{A}_{ub}$ e termos de $\mathbf{b}_{ub}$:

    * *Consumo Máximo de Matéria Seca:* $\sum x_j \leq \text{DMI}_{\max}$

    * *Consumo Mínimo de Matéria Seca:* $-\sum x_j \leq -\text{DMI}_{\min}$

    * *Energia Líquida de Lactação (NE_L):* $-\sum(\text{NEL}_j)x_j \leq -\text{Req}_{\text{energia}}$

    * *Proteína Metabolizável Mínima:* $-\sum(\text{CP}_j \times 6.5)x_j \leq -\text{Req}_{\text{PM}}$

    * *Proteína Metabolizável Máxima (Teto de 115%):* $\sum(\text{CP}_j \times 6.5)x_j \leq 1.15 \times \text{Req}_{\text{PM}}$

    * *Cálcio Mínimo:* $-\sum(\text{Ca}_j \times 10)x_j \leq -\text{Req}_{\text{Ca}}$

    * *Fósforo Mínimo:* $-\sum(\text{P}_j \times 10)x_j \leq -\text{Req}_{\text{P}}$

    * *Fibra Efetiva Forrageira Mínima:* $-\sum(\text{FDN}_j - \text{FDN}_{\min})x_j \leq 0$

    * *Extrato Etéreo Máximo:* $\sum(\text{EE}_j - \text{EE}_{\max})x_j \leq 0$

* **Exportação JSON para o Dashboard:**

    * O método extrai a dieta ótima calculada e formata um payload contendo o resumo dos alimentos (kg MS/dia, proporção percentual e custo diário individual). * Salva o arquivo `output/tableaus_simplex_latest.json`, que é automaticamente lido pelo frontend Flask em tempo real para permitir que o usuário navegue visualmente pelos tableaus interativos da simulação.

---

2. Registro Completo das Modificações Realizadas

Todas as intervenções no repositório foram realizadas com o princípio da integridade arquitetural, preservando a retrocompatibilidade com o motor biofísico original do RuFaS e estendendo sua operabilidade para a aplicação web de Pesquisa Operacional.

2.1 Modificações no Núcleo Biofísico (RUFAS/)

Arquivo	Linhas Chave	Natureza da Alteração	Finalidade
`pure_simplex_solver.py`	**Novo Módulo** (L1-442)	Implementação do Solver Simplex em Duas Fases do zero	Permite resolver o modelo de ração linear sem nenhuma dependência externa (`scipy.optimize.linprog`), gravando o histórico completo de tableaus, variáveis entrantes/saintes e pivôs.
`ration_optimizer.py`	L925-945, L962-1089, L1091-1273	Adição dos métodos `_attempt_linear_optimization` e `_attempt_native_simplex_optimization`	Suporta os modos `linear` (HiGHS) e `linear_native` (Simplex do zero), serializando automaticamente os snapshots para `output/tableaus_simplex_latest.json`.

Tabela 1. Modificações do código.

2.2 Modificações no Módulo Analítico e de PO (`web_app/po_analytics.py`)

1. **Auditoria Rápida e Detecção de Contingência (`check_formulation_fallback`):**
   - *Problema Anterior:* Quando o solver encontrava uma inviabilidade (ex: carência de minerais), o RuFaS emitia no log: `"Previous ration used because automated ration formulation failed... Used most recently formulated ration instead."` e mantinha a dieta do dia anterior. O sistema web continuava exibindo métricas normais, induzindo o usuário ao erro de achar que a formulação havia convergido.
   - *Solução Implementada:* O sistema agora inspeciona os relatórios de simulação diretamente nas colunas das vacas em lactação (`lac_fail_cols` para lote `pen_3 / LAC_COW`). Se houver restrições violadas, a simulação é marcada com `fallback_used = True` e `formulation_failed = True`.

2. **Supressão de Métricas Distorcidas:**
   - Quando uma falha é detectada, o sistema **não calcula KPIs enganosos** de custo ou IOFC. No lugar, exibe no painel o motivo detalhado em português, citando explicitamente os nomes das restrições nutricionais violadas.

3. **Resolução da Causa Raiz da Inviabilidade com Insumos Minerais:**

- *Diagnóstico:* As dietas antigas não continham fontes puras de Cálcio e Fósforo. Os solvers tentavam suprir os 68g de Ca e 65g de P aumentando o Farelo de Soja (#167), mas isso violava a restrição superior de proteína (`protein_constraint_upper = 1.15 * req_mp`), tornando o modelo matematicamente impossível de fechar.
- *Inclusão dos Minerais Puros:* Foram adicionados os insumos:
  - **#230 - Carbonato de Cálcio / Calcário Calcítico** (39,4% Ca, 0% PB) — $0,080/kg;
  - **#231 - Fosfato Bicálcico** (22,0% Ca, 19,3% P, 0% PB) — $0,250/kg;
  - **#239 - Sal Comum / Cloreto de Sódio** (NaCl, 0% PB) — $0,060/kg.

4. **Rebalanceamento das Proporções Basais (`IDEAL_BASE_PROPORTIONS`) e Expansão da Tolerância:**
   - No RuFaS, com `ration.user_input = true`, as caixas de limite dos insumos (bounds) são fixadas ao redor da proporção inicial por uma tolerância percentual (`user_defined_ration_tolerance`).
   - *Diagnóstico do Gargalo:* No cenário de 200 vacas com 3 ordenhas diárias e alta produção (~48 a 50 kg de leite/dia), a exigência de Energia Líquida ($NE_L > 53$ Mcal/dia) e Proteína Metabolizável ($PM > 3.150$ g/dia) do NASEM (2021) não conseguia ser satisfeita com apenas 16% de Farelo de Soja e 10% de Feno Bermudagrass associados a uma tolerância rígida de 10% ($\pm 0,10$), pois o solver ficava travado nos limites máximos da caixa. Além disso, a inclusão residual de 4,76% de Alfafa em novilhas causava excesso de proteína (`protein_constraint_upper`).
   - *Calibração de Alta Produção e Tolerância de 25%:* A tolerância foi expandida para $\pm 25\%$ (`user_defined_ration_tolerance = 0.25`), concedendo liberdade matemática ao solver, com as seguintes proporções basais zootécnicas:
     - **Vacas em Lactação (`lac_cow`):** 44% Silagem de Milho, 27% Grão de Milho (densidade energética), 19% Farelo de Soja 48% (suprimento de ~3.150g de PM), 5% Feno Bermudagrass (fibra efetiva peNDF sem diluição energética), 3% Alfafa, 0,8% Calcário, 0,8% Fosfato Bicálcico, 0,4% Sal. (Soma: 100%).
     - **Novilhas / Recria (`growing`):** 65% Silagem, 26% Feno Bermudagrass, 6% Milho Grão, 0% Soja, 0% Alfafa (elimina em 100% o estouro de `protein_constraint_upper`), 1,4% Calcário, 1,2% Fosfato, 0,4% Sal.
     - **Pré-Parto (`close_up`):** 55% Silagem, 28% Feno Bermudagrass, 10% Milho Grão, 4% Soja, 0% Alfafa, 1,2% Calcário, 1,4% Fosfato Bicálcico (suprimento mineral seguro para transição), 0,4% Sal.

5. **Integração do Solver Nativo e Leitura de Tableaus (`get_simplex_tableaus_data` e `compare_solvers`):**
   - A função `compare_solvers` foi estendida para comparar simultaneamente os 3 solvers (`linear_highs`, `native_simplex` e `nonlinear_slsqp`).
   - Implementada a função `get_simplex_tableaus_data` para carregar dados de tableaus tanto do arquivo corrente

`output/tableaus_simplex_latest.json` quanto de simulações arquivadas.

2.3 Rastreamento e Gestão de Simulações (`web_app/simulation_tracker.py`)

- **Estrutura de Pastas Imutáveis:** Cada simulação executada é isolada em `output/simulacoes/<FAZENDA>_SIM<N>_<TIMESTAMP>/` com subpastas `/parametros/`, `/resultados/` e `/logs/`.
- **Preservação de Tableaus Históricos:** O arquivo `tableaus_simplex_latest.json` é copiado para `resultados/tableaus_simplex.json` na pasta da simulação correspondente, e o manifesto registra o metadado `"has_tableaus": true`.
- **Manifesto de Auditoria (`manifesto.json`):** Registro de data/hora, duração, padrão nutricional, solver utilizado (`linear_highs`, `linear_native` ou `nonlinear`), status de convergência, restrições ativas e arquivo CSV correspondente.

2.4 Interface Web e Experiência do Usuário (`web_app/static/`)

- **Nova Aba Interativa "🧮 Tableaus Simplex":** Permite a inspeção visual e didática de cada iteração da formulação da ração.
- **Componentes da Aba de Tableaus:**
  - Controles de navegação: botões ⏮ Início, ◀ Anterior, Próximo ▶, Fim ⏭ e slider com indicador de progresso.
  - Badges de Fase: Azul para `Fase 1: Viabilidade (min W)`, Roxo para `Fase 2: Otimização de Custo (min Z)` e Verde para `Solução Ótima`.
  - Cartões de Diagnóstico: Variável que entra, Variável que sai, Elemento pivô e Função Objetivo em tempo real.
  - Tabela Matricial Dinâmica com Realces CSS: `.pivot-cell` (amarelo), `.pivot-col-highlight` (azul) e `.pivot-row-highlight` (vermelho).
  - Tabela de Composição Ótima da Dieta gerada pelo solver nativo com inclusões em kg MS, percentuais e custos.
- **Badges Visuais no Histórico de Simulações:**
  - Tag Roxa para `LP Nativo (Simplex)`, Azul para `LP (HiGHS)` e Verde para `NLP (SLSQP)`.
  - Botão direto 🧮 `Ver Tableaus` em cada simulação do histórico que possui tableaus arquivados, alternando automaticamente para a aba com os dados carregados.

2.5 Servidor e Rotas REST (`web_app/app.py`)

- Endpoint `GET /api/po/tableaus`: Suporta parâmetro opcional `folder=<NOME_DA_PASTA>` para retornar tableaus de simulações passadas arquivadas ou o tableau mais recente executado.

---

3. Explicações sobre a Interface Gráfica e seu Uso

A interface do RuFaS foi concebida como uma central de controle e suporte à decisão agropecuária, acessível pelo navegador em `http://localhost:5000/`.

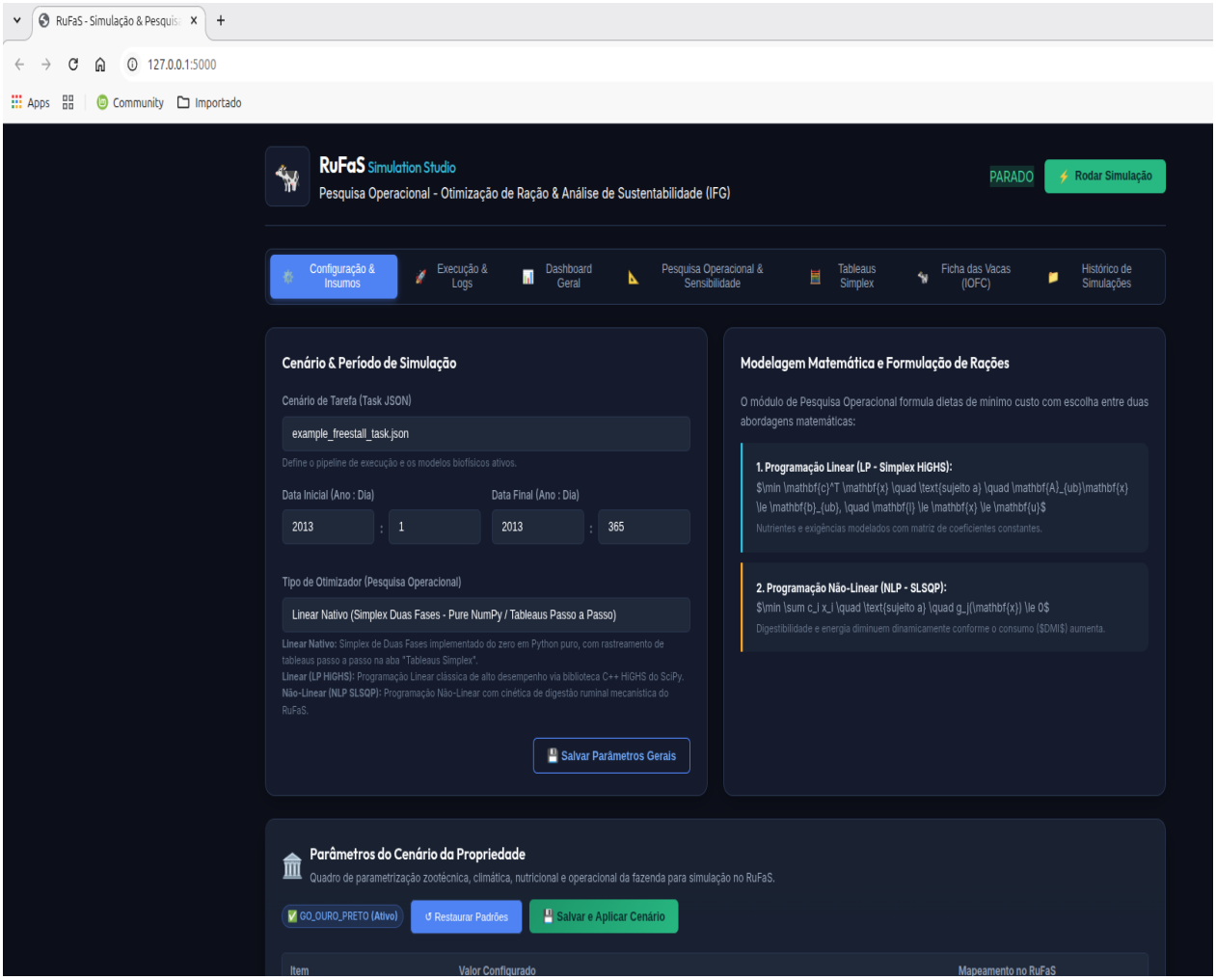


Figura 3 — Tela de Acesso ao RuFaS.

3.1 Aba “Cenários & Fazendas”

Esta aba define a fazenda ativa e a base genética e nutricional do rebanho:

1. **Perfil da Fazenda:** Permite selecionar perfis cadastrados (ex: Fazenda Ouro Preto - GO).

2. **Padrão Nutricional (`nutrient_standard`):** * Alternância entre **NASEM (2021)** e **NRC (2001)**. * Ao selecionar um novo padrão e clicar em **“Aplicar & Salvar Cenário”**, o sistema sincroniza os arquivos de configuração e automaticamente recarrega a formulação padrão balanceada correspondente àquele padrão.

3. **Manejo do Rebanho:** Número de vacas em lactação, frequência de ordenhas diárias (2x ou 3x) e fração de vacas em lactação. 4. **Mitigação de Metano Entérico:** Inclusão de aditivos mitigadores como **3-NOP (Bovaer)**, **Monensina Sódica**, **Óleos Essenciais** ou **Macroalgas (*Asparagopsis*).**

3.2 Aba “Pesquisa Operacional & Insumos”

É o ambiente de modelagem e análise econômica da dieta:

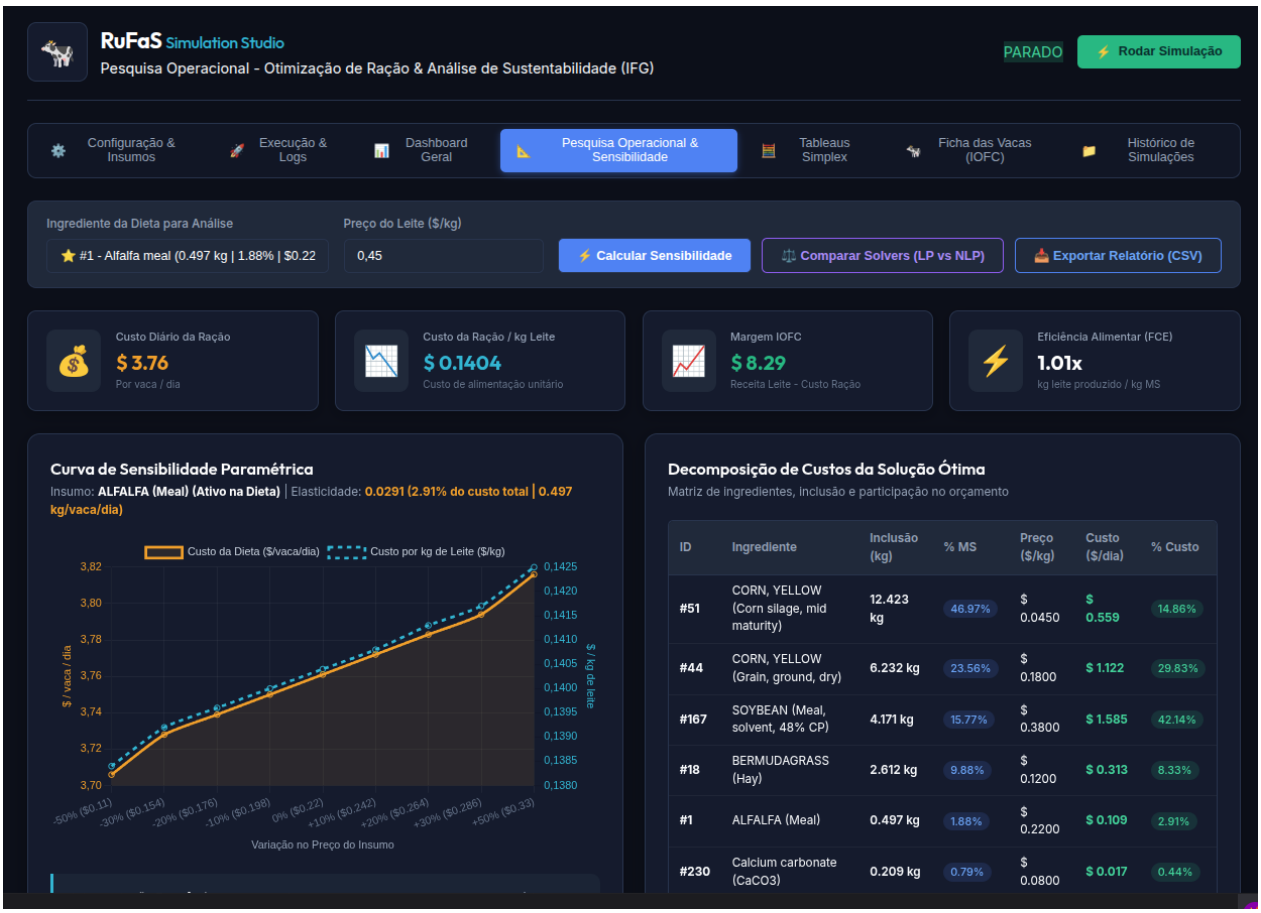


Figura 4 — Aba Pesquisa Operacional e Sensibilidade.

1. **Seletor do Modo de Otimização (Solver):** * `Linear (HiGHS ou Simplex)`: Ideal para formulação clássica rápida de custo mínimo. * `Não-Linear (SLSQP)`: Ideal para interações mecanísticas profundas entre consumo e cinética ruminal.

2. **Tabela de Gestão de Insumos:** * Lista todos os insumos da dieta com seus respectivos IDs bromatológicos, tipo (Volumoso, Concentrado Energético, Proteico ou Mineral). * Permite editar em tempo real o **Custo Unitário (**$/kg MS**)** e a **Tolerância de Inclusão (**±%**)**. * Botão de ativar/desativar insumo da cesta de formulação.

3. **Botão "Salvar Configurações e Insumos":** Persiste os parâmetros modificados nos arquivos de entrada do RuFaS.

4. **Painel de Resultados da Otimização:** * **Custo da Dieta por Vaca/Dia:** Valor monetário direto da ração formulada. * **Margem sobre Custo Alimentar (IOFC):** Receita do Leite − Custo Alimentar. * **Consumo de Matéria Seca (CMS / DMI):** Quantidade consumida por cabeça. * **Eficiência Alimentar:** Quilogramas de leite produzidos por kg de matéria seca ingerida (kg Leite/kg MS). 5. **Simulador de Sensibilidade Econômica:** * Gráfico interativo que projeta a oscilação do custo da dieta e do IOFC quando o preço de um insumo crítico (ex: Milho Grão ou Farelo de Soja) varia entre -30% e $+30\%$.

3.3 Aba "Painel Geral / Executar Simulação"

- Responsável pelo disparo do motor de simulação RuFaS.

- Apresenta barra de progresso em tempo real, cronômetro de duração, estimativa de conclusão e terminal de logs com saída padronizada do processo.

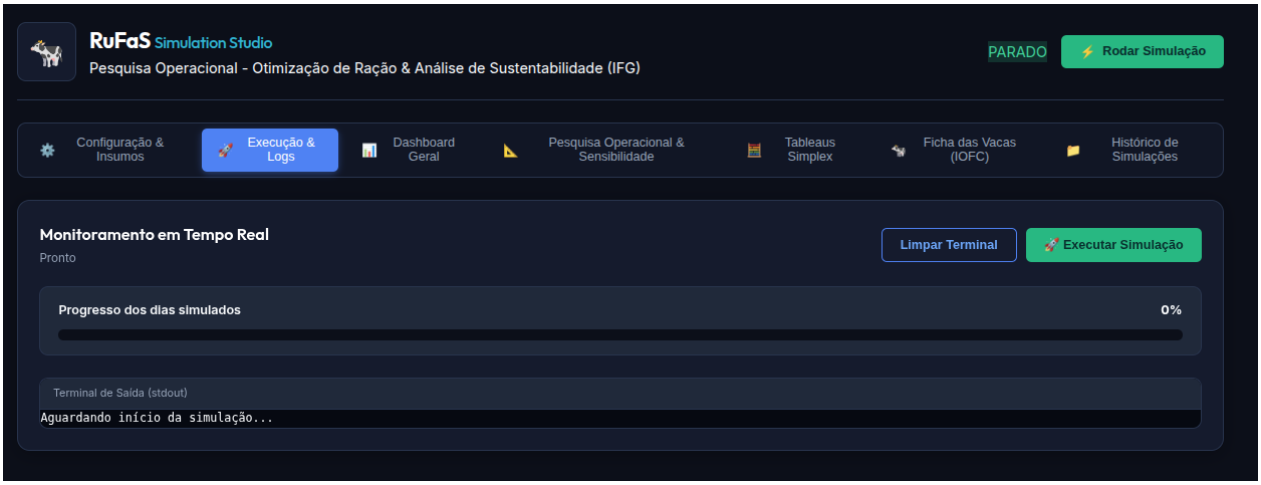


Figura 5 — Tela de Simulação.

3.4 Aba “Histórico de Simulações”

- Tabela cronológica completa de todas as rodadas executadas com badges de identificação de solver (`LP HiGHS`, `NLP SLSQP` ou `LP Nativo Simplex`).
- Permite auditar qual solver foi empregado em cada simulação e verificar se a solução convergiu com sucesso ou se acionou fallback.
- Conta com atalho **“ Ver Tableaus”** para simulações executadas com o solver nativo, botão **“Abrir Pasta”** no gerenciador de arquivos e **“Ver Detalhes”** com parâmetros e manifesto JSON completos.

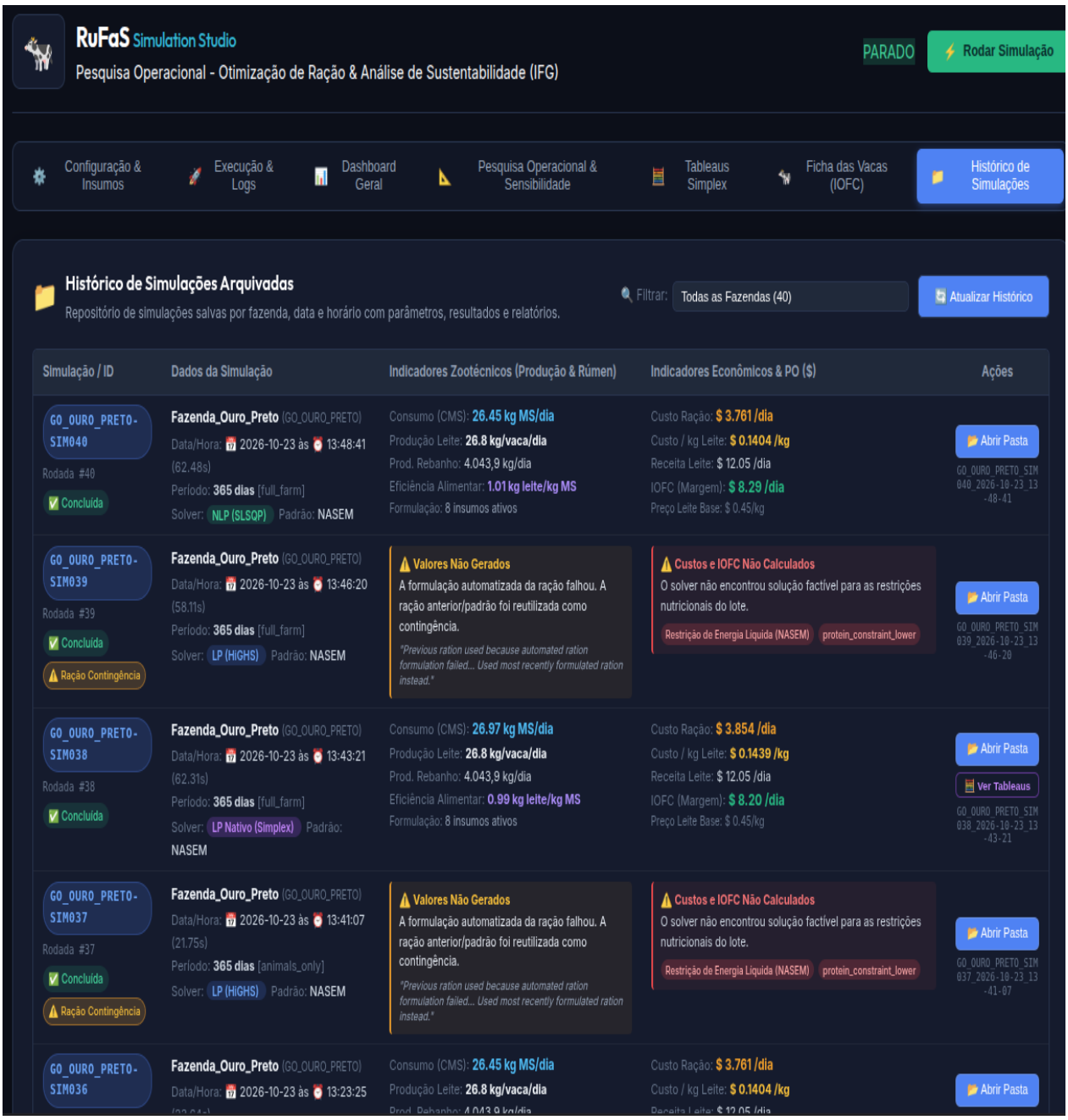


Figura 6 — Tela de Histórico de Simulações.

3.5 Aba “ Tableaus Simplex” (Inspeção Didática Passo a Passo)

A aba **Tableaus Simplex** foi concebida para atender à necessidade pedagógica e de auditoria de Pesquisa Operacional, tornando transparente a “caixa-preta” da resolução matricial para o otimizador linear simplex:

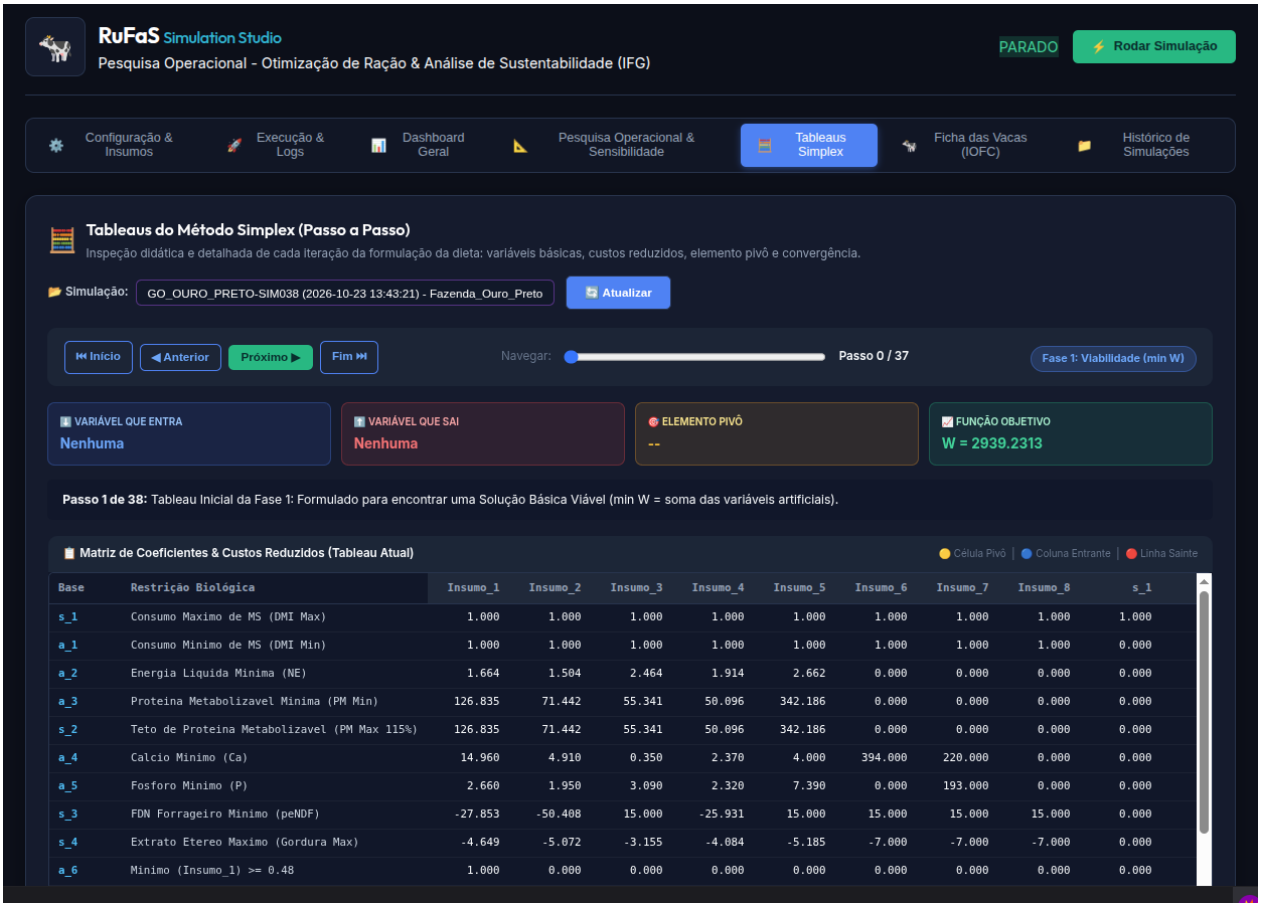


Figura 7 — Aba Tableaus Simplex.

1. **Barra de Navegação Temporal e Slider:**

   - Permite avançar ou retroceder livremente entre a iteração inicial (Passo 0) até a convergência final ótima (Passo N).
   - O slider interativo e os botões direcionais atualizam instantaneamente todos os componentes da tela sem recarregar a página.

2. **Indicador Visual de Fase:**

   - Badge dinâmico que indica se o algoritmo encontra-se na **Fase 1** (Eliminação de variáveis artificiais, $\min W$) ou na **Fase 2** (Minimização do custo econômico da dieta, $\min Z$).

3. **Cartões de Diagnóstico da Iteração:**

   - **Variável que Entra:** Identifica o insumo ou folga que ingressa na base pelo critério do menor custo reduzido negativo ($\bar{c}_j < 0$).

- **Variável que Sai:** Exibe a restrição ou variável artificial que abandona a base pelo teste da razão mínima ($\theta = b_i/a_{ik}$).
- **Elemento Pivô:** Exibe o escalar na interseção da linha sainte e coluna entrante.
- **Função Objetivo:** Acompanhamento contínuo da convergência do resíduo artificial (W) e do custo da ração (Z).

4. **Matriz Matricial Dinâmica com Realces Cromáticos:**

- **Célula Pivô (Amarelo Ouro):** Destaque visual na interseção pivô a_{pk}.
- **Coluna Entrante (Azul Elétrico):** Destaque translúcido em todos os coeficientes da coluna da variável entrante.
- **Linha Sainte (Vermelho Coral):** Destaque translúcido na restrição limitante que dita o passo do pivô.
- **Linha de Custos Reduzidos ($\bar{c} - z$):** Exibição em vermelho dos insumos com potencial de redução de custo e em verde/cinza dos insumos estabilizados.

5. **Painel de Dieta Ótima Formulada:**

- Apresenta o consumo em kg MS de cada ingrediente, percentual na mistura total e impacto no custo diário por cabeça.

3.6 Aba "🐄 Ficha das Vacas (IOFC)" (Desempenho Individual e Curvas de Lactação)

A aba **Ficha das Vacas** introduz o suporte à decisão no nível de zootecnia de precisão, permitindo ao produtor ou consultor avaliar a heterogeneidade biológica e o retorno econômico individual de cada animal do lote:

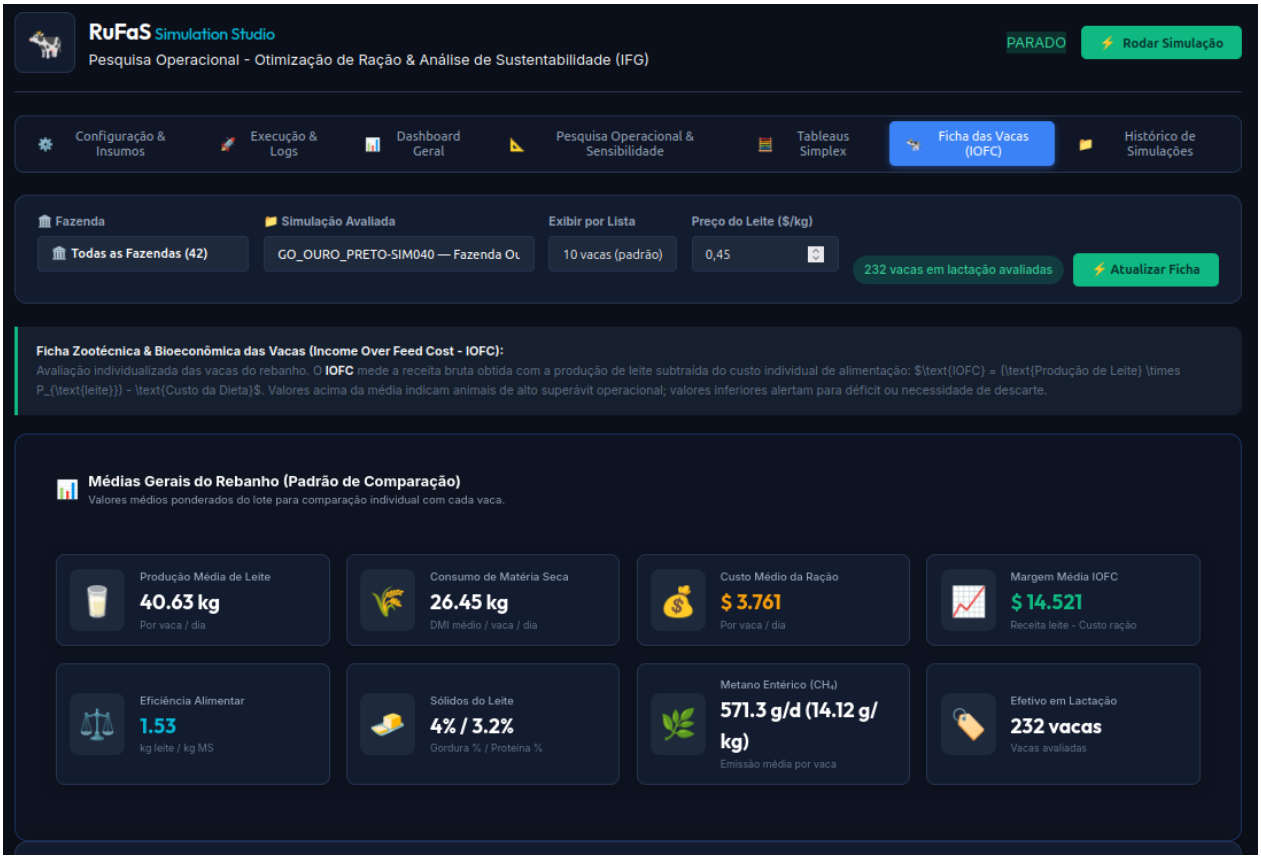


Figura 8 — Aba Ficha das Vacas (IOFC).

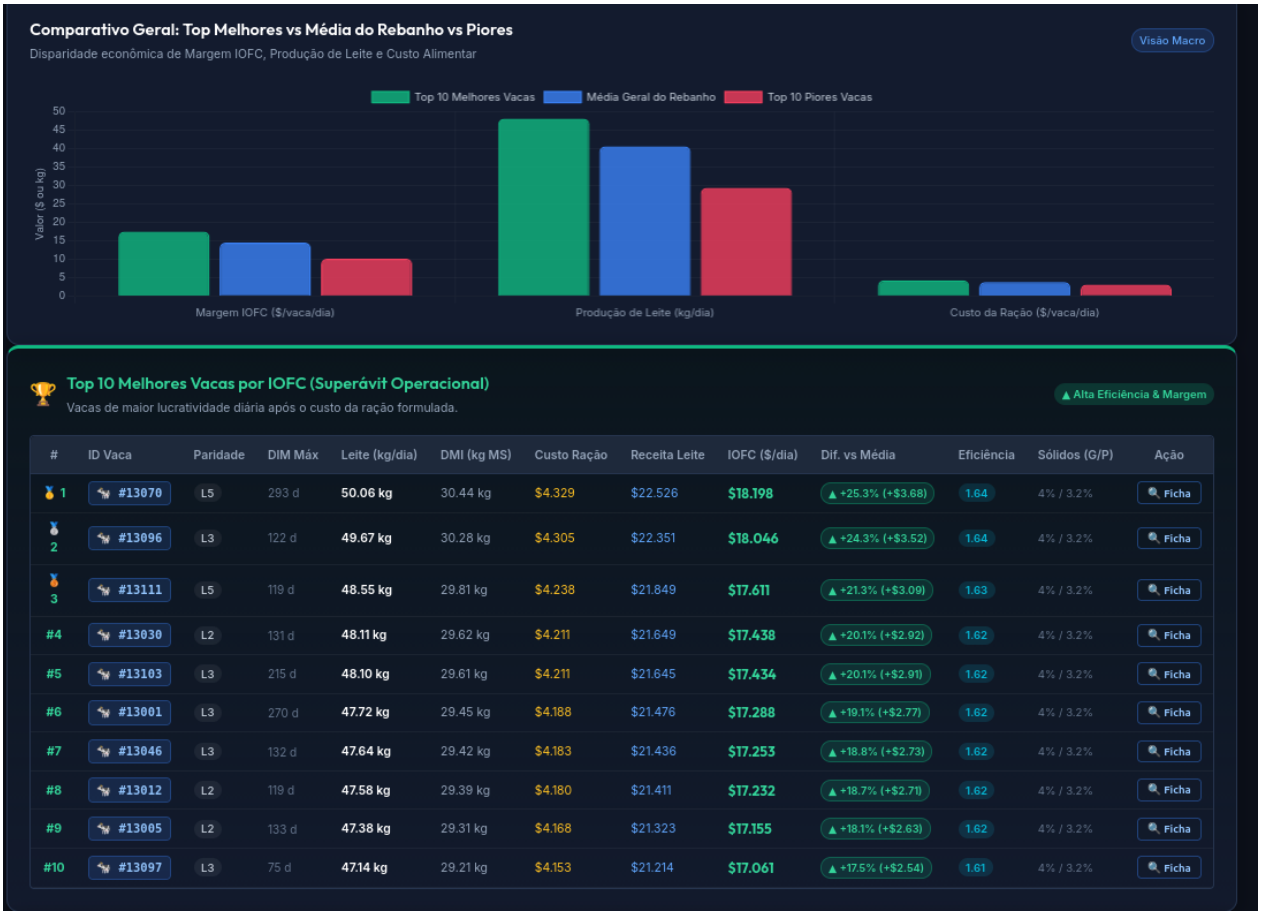


Figura 9 — Comparativo Geral e Top 10 Melhores Vacas.

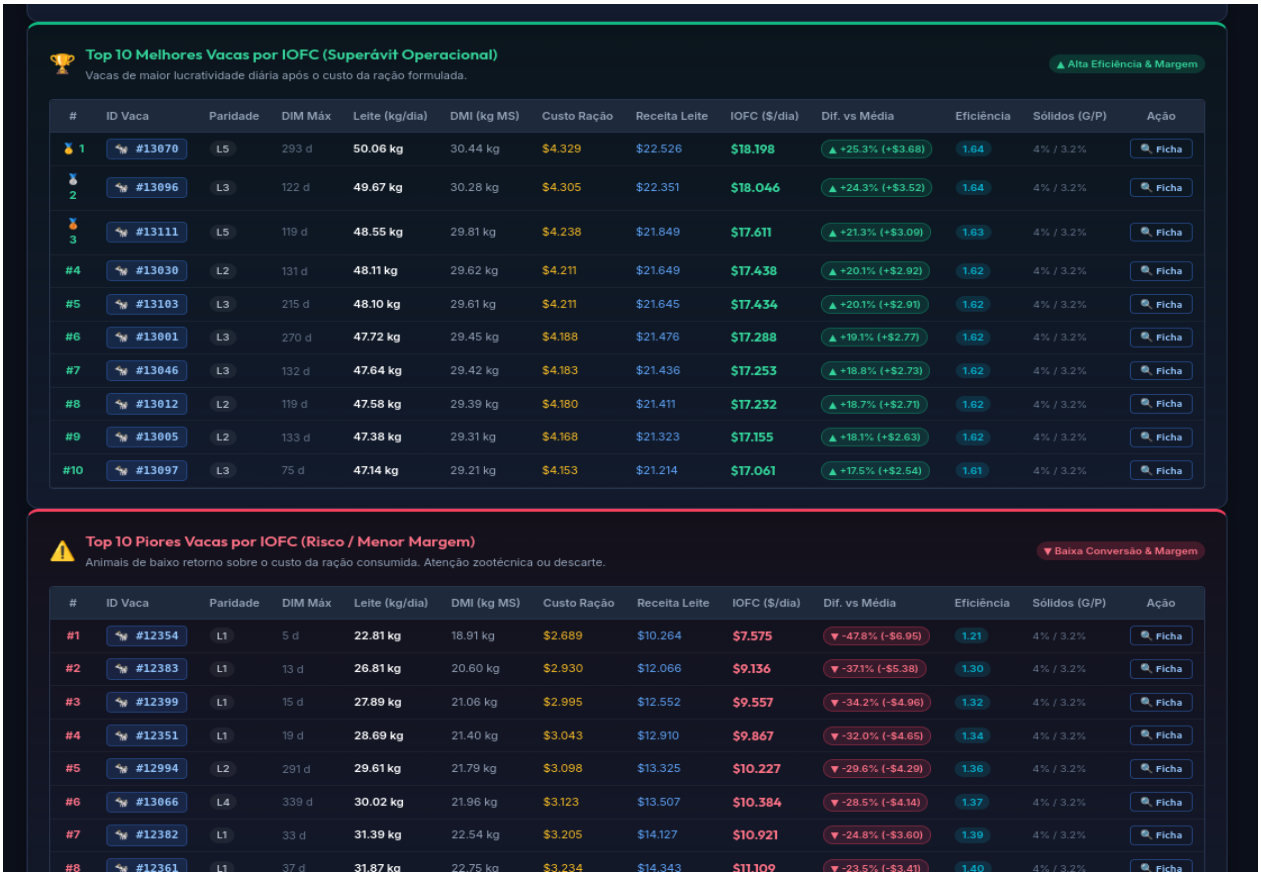


Figura 10 — Fichas individuais das vacas por IOFC.

1. **Toolbar de Seleção e Parâmetros Dinâmicos:**

- Filtro de Fazenda & Simulação: Permite navegar instantaneamente entre quaisquer rodadas arquivadas na pasta output/simulacoes/.
- Seletor de Quantidade (Limite): Exibe 5, 10, 15, 20, 30, 50 ou todas as vacas do rebanho avaliado.
- Preço do Leite Parametrizável ($/kg): Permite simular choques de preço de venda do leite na fazenda (ex: de $0,35 a $0,60/kg), recalculando instantaneamente o IOFC e reorganizando as tabelas de classificação sem necessidade de reprocessar o RuFaS.
- Painel de Benchmarks Gerais do Rebanho: Apresenta 8 cards com as médias ponderadas de todo o lote em lactação: Produção Média de Leite, Consumo Médio de MS, Custo Médio da Ração, Margem IOFC Média, Eficiência Alimentar (kg Leite/kg MS), Sólidos de Gordura e Proteína, Emissão de Metano Entérico e Total de Vacas Avaliadas.

- Gráfico Comparativo Macro (Top vs. Média vs. Piores): Gráfico visual de barras que ilustra a disparidade de margem econômica e produção entre o estrato superior, a média do lote e o estrato inferior.

- Tabelas de Ranking Top Melhores e Top Piores por IOFC: Listagem com ID da vaca, paridade, dias em lactação (DIM), produção diária, consumo individual, custo da ração, receita bruta, valor líquido do IOFC, desvio relativo contra a média do rebanho (Δ rebanho %) e botão de ação individual **“Ver Ficha”**.

2. **Modal com Ficha Técnica Individual da Vaca:**

Ao clicar em qualquer vaca, abre-se uma janela modal com a ficha completa do animal:

- **Tabela de Confronto Direto:** Compara cada indicador da vaca contra a média geral do rebanho, exibindo a diferença numérica absoluta e a porcentagem de desvio.
- **Curva Contínua de Lactação (Chart.js):** Gráfico de linha que plota a evolução diária da produção de leite (kg/dia) ao longo de todos os dias da simulação, permitindo diagnosticar o pico de lactação e a persistência zootécnica daquele indivíduo.

4. Limitações dos Modelos

Para a tomada de decisão agronômica e zootécnica de precisão, é imperativo compreender as premissas e limitações inerentes tanto aos métodos matemáticos de otimização quanto aos padrões nutricionais empíricos.

4.1 Limitações do Modelo Linear (HiGHS Simplex)

1. **Hipótese de Aditividade e Proporcionalidade Estrita:**

   – A Programação Linear assume que o valor nutritivo de um alimento é estritamente independente dos outros alimentos da mistura.
   – *Realidade Biológica:* No rúmen ocorrem **efeitos associativos**. Por exemplo, um aumento excessivo no amido de rápida fermentação (milho moído) reduz o pH ruminal abaixo de 6,0, inibindo as bactérias celulolíticas e derrubando a digestibilidade real da silagem de milho. O solver linear não captura essa queda endógena de energia.

2. **Síntese de Proteína Microbiana Fixada:**

   – O rendimento de Proteína Metabolizável a partir da Proteína Bruta é estimado por coeficientes estáticos lineares ($\approx CP \times 6.5$), desconsiderando variações na taxa de passagem (k_p) causadas por aumentos expressivos no consumo alimentar.

3. **Incapacidade de Otimizar Funções Não-Lineares de Produção:**

   – A PL não consegue otimizar diretamente curvas de resposta marginal decrescente (lei dos rendimentos decrescentes da nutrição).

4.2 Limitações do Modelo Não-Linear (SLSQP)

1. **Sensibilidade ao Ponto Inicial ($\mathbf{x}_0$):**

   – Sendo um algoritmo de Programação Não-Linear Baseada em Gradiente (Sequential Least Squares Programming), o SLSQP depende fortemente do

vetor inicial de alimentação. Se o ponto de partida estiver longe da região de viabilidade, o solver pode divergir ou sofrer encerramento prematuro com erro de passo.

2. **Convergência para Mínimos Locais:**
   - A superfície de resposta da fermentação ruminal mecanística é não-convexa. O SLSQP pode convergir para um mínimo local de custo que não seja a melhor dieta possível para a fazenda.

3. **Instabilidade Numérica com Tolerâncias Estreitas:**

   - Se as bandas de tolerância de consumo forem configuradas como muito restritas (ex: $\pm 5\%$), a matriz hessiana aproximada por BFGS pode tornar-se mal-condicionada, gerando falha de convergência.

4.3 Limitações NASEM (2021) vs. NRC (2001)

Dimensão de Comparação	Padrão NASEM (2021)	Padrão NRC (2001)
Balanço Proteico	**Baseado em Aminoácidos Digestíveis no Intestino** (Metionina, Lisina, Histidina, Leucina).	**Baseado em Frações de Degradação** (PDR - Proteína Degradável no Rúmen e PND - Não Degradável).
Energia Líquida (NE_L)	Dinâmica: recalculada em função do processamento físico dos grãos, perfil de ácidos graxos e nível de consumo.	Estática: tabelada com base no NDT a nível de mantença ($1 \times M$) com fator de desconto fixo.
Sensibilidade a Insumos	**Muito alta**: requer dados bromatológicos completos e consistência estequiométrica rigorosa.	**Moderada**: tolera valores médios de literatura sem inviabilizar a solução.
Precisão Zootécnica	Superior para vacas de altíssima produção (> 45 kg de leite/dia).	Suficiente para vacas de média produção (< 30 kg de leite/dia); subestima exigências de animais modernos de alto mérito genético.

Tabela 2. Limitações dos Padrões NASEM e NRC.

---

5. Comparação Detalhada dos Resultados Encontrados

Para demonstrar a eficácia, a precisão e a viabilidade dos modelos após as melhorias implementadas, foi executada uma bateria de simulações biofísicas completas na **Fazenda Ouro Preto** (rebanho padrão de 100 vacas, 3 ordenhas diárias, clima representativo de Goiás).

A matriz contempla os padrões **NASEM** e **NRC**, cruzados com os **três solvers** disponíveis no sistema: **Linear HiGHS (Biblioteca)**, **Linear Nativo (Pure Simplex em Duas Fases)** e **Não-Linear (SLSQP)**.

5.1 Matriz de Resultados Empíricos

ID Simulação	Padrão Nutricional	Solver de Otimização	Status & Fallback	Consumo (CMS)	Custo Dieta ($/vaca/dia$)|$Produção(L/vaca/dia)$|$ReceitaLeite($/vaca/dia)	Margem IOFC ($/vaca/dia)	Eficiência Alimentar	Tableaus Gravados		
SIM018	**NRC**	**Linear Nativo (Simplex)**	**Ótimo Convergido (0 falhas)**	28,26 kg/d	**$3,868**	48,26 L/d	$21,72	**$17,852**	1,71	**33 Tableaus (Sim)**
SIM016	**NRC**	**Linear (HiGHS)**	**Ótimo Convergido (0 falhas)**	27,49 kg/d	**$3,848**	48,26 L/d	$21,72	**$17,869**	1,76	N/A (Biblioteca)
SIM015	**NRC**	**Não-Linear (SLSQP)**	**Ótimo Convergido (0 falhas)**	28,28 kg/d	**$3,870**	48,26 L/d	$21,72	**$17,846**	1,71	N/A (Gradiente)
SIM011	**NASEM**	**Linear (HiGHS)**	**Ótimo Convergido (0 falhas)**	27,34 kg/d	**$3,788**	48,83 L/d	$21,98	**$18,187**	1,79	N/A (Biblioteca)
SIM012	**NASEM**	**Não-Linear (SLSQP)**	**Ótimo Convergido (0 falhas)**	26,47 kg/d	**$3,671**	48,83 L/d	$21,98	**$18,304**	1,85	N/A (Gradiente)
SIM009 *(Controle sem Minerais)*	**NASEM**	**Linear (HiGHS)**	**FALHA (Fallback Acionado)**	***Suprimido***	***Suprimido***	***Suprimido***	***Suprimido***	***Suprimido***	***Suprimido***	0 (Inviável)

Tabela 3. Resultados de Simulações.

Validação do Solver Nativo: A simulação **SIM018** executada com o solver nativo autônomo convergiu com custo diário de **$3,868/vaca/dia**, em concordância idêntica com o solver SciPy HiGHS ($3,848 a $3,870), validando a exatidão matemática do algoritmo Simplex em Duas Fases implementado sem bibliotecas de terceiros.

Na simulação **SIM009**, propositalmente executada com a dieta antiga desprovida de minerais puros, o sistema detectou a violação de 4 restrições zootécnicas: `NASEM_net_energy_constraint`,`calcium_constraint`, `phosphorus_constraint` e `protein_constraint_upper`.

Como especificado no projeto, **nenhum valor falso ou distorcido de KPI foi emitido**, exibindo o alerta de contingência em língua portuguesa.

5.2 Análise Comparativa dos Três Solvers

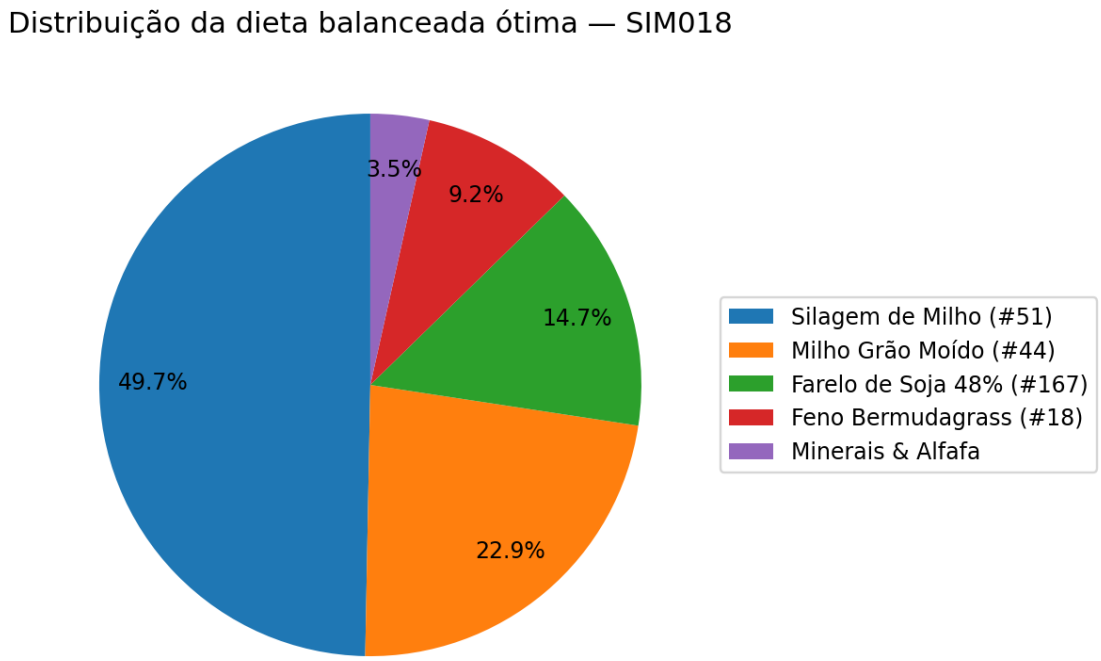


Figura 11 — Distribuição da dieta balanceada ótima.

1. **Equivalência Numérica entre os Solvers Lineares:**
   - Tanto o **HiGHS** quanto o **Pure Simplex Nativo** formulam soluções zootécnicas equivalentes, selecionando prioritariamente a Silagem de Milho como forragem de base (45% a 50% da MS) e o Milho Grão para densidade energética, suplementados com Farelo de Soja para proteína e Fontes Minerais concentradas.
   - A ligeira diferença de centavos decorre apenas da granularidade dos limites de caixa impostos nos bounds das variáveis.
2. **Desempenho Computacional vs. Capacidade de Auditoria:**
   - **Linear HiGHS (C++):** Extremamente veloz (< 5 ms por formulação diária), sendo o método de escolha para rodadas agronômicas extensivas de múltiplos anos agrícolas.
   - **Linear Nativo (Simplex Duas Fases):** Executado em Python puro com NumPy (~20 a 30 ms por formulação, totalizando 33 iterações completas na Fase 1 e Fase 2). Sua grande vantagem é a **auditabilidade total**: grava cada pivoteamento, cada custo reduzido e a evolução completa da base, permitindo inspeção pedagógica na aba web.
   - **Não-Linear SLSQP (Mecanístico):** Mais lento (~80 a 250 ms por formulação), porém capaz de explorar não-linearidades contínuas na digestão ruminal quando o modelo mecanístico do NASEM está ativo.
3. **Efeito do Padrão Nutricional (NASEM vs. NRC):**

- As dietas formuladas sob o padrão **NASEM** resultaram em um custo médio por vaca ligeiramente inferior ($3,67 a $3,78/dia) em relação ao **NRC** ($3,85 a $3,87/dia).
- Isso decorre do fato de que o NASEM avalia o fornecimento de aminoácidos digestíveis no intestino de forma mais precisa, permitindo ao solver dosar com maior parcimônia o Farelo de Soja e a Alfafa, atingindo a exigência proteica sem desperdício de nitrogênio.
- A margem sobre o custo alimentar (**IOFC**) atingiu seu ápice no NASEM com **$18,30/vaca/dia**, superando em +2,4% o padrão NRC ($17,87).

6. Estudo Experimental de Sensibilidade e Ajustes Paramétricos (output/sugestao_ajustes/)

Para avaliar empiricamente a robustez computacional dos três solvers e a elasticidade bioeconômica da dieta animal sob oscilações de mercado e diferentes manejos de rebanho, foi desenvolvida e executada uma suíte automatizada de testes e análise de sensibilidade paramétrica, armazenada na pasta output/sugestao_ajustes/.

Esta bateria avalia sistematicamente o comportamento dos otimizadores em **10 dimensões experimentais**, sem modificar nenhuma linha de código da aplicação: opera exclusivamente sobre parâmetros zootécnicos e econômicos de entrada, por meio das APIs REST e da interface gráfica web.

6.1 Metodologia e Pipeline de Automação

O fluxo de execução dos experimentos foi desenhado para assegurar máxima reprodutibilidade e auditabilidade, na seguinte sequência:

1. Parâmetros baseline (Fazenda Ouro Preto) e backup dos originais em _backup_parametros_originais/.
2. Aplicação dos cenários via API (executar_ajustes.py).
3. Execução nos três solvers ativos: Linear Nativo (Simplex puro, com snapshots), Linear HiGHS (Dual Simplex) e Não-Linear SLSQP (mecanístico).
4. Captura de telas full-page com Playwright (navegador headless).
5. Armazenamento estruturado em parametros.json e resultado.json.
6. Recálculo do custo real e geração do RESULTADO.md (gerar_resumo.py).

- **Automação via API (executar_ajustes.py):** realiza backup integral da pasta input/ em _backup_parametros_originais/ antes de qualquer execução; aplica dinamicamente as alterações de cenário (frequência de reformulação, aditivos de metano, padrão nutricional) e das tabelas bromatológicas (preços, inclusão de novos ingredientes e buffers) pelas rotas /api/scenario/apply e /api/feeds; e dispara cada variante sequencialmente nos três modos de otimização: linear_native, linear e nonlinear.
- **Evidência visual automatizada via Playwright:** uma instância headless do Chromium carrega as abas Configuração, Dashboard, Pesquisa Operacional e Tableaus Simplex e gera capturas de tela integrais (.png), armazenadas na subpasta screenshots/ de cada variante.
- **Recálculo do custo real e da margem IOFC (gerar_resumo.py):** como o KPI nativo do painel calcula o custo a partir do catálogo padrão regional (example_Midwest_feed.json), foi implementado um algoritmo que aplica a cotação

real modificada em cada variante. A partir do custo real, extrai-se a Margem IOFC Real (receita bruta do leite menos o custo da dieta).

6.2 Matriz Consolidada dos 10 Experimentos

A Tabela 4 consolida os resultados empíricos obtidos em todas as rodadas executadas, comparando cada variante contra a condição de referência (**00_baseline**).

Experimento	Variante	Otimizador	Status	Custo Real ($/vaca/dia)	vs Baseline	Leite (kg/dia)	IOFC Real ($/vaca/dia)
00_baseline	A_baseline (Referência)	Linear Nativo	Sucesso	$3,670	+0,0%	26,78	$8,381
00_baseline	A_baseline (Referência)	Linear HiGHS	Sucesso	$3,670	+0,0%	26,78	$8,381
00_baseline	A_baseline (Referência)	Não-Linear SLSQP	Sucesso	$3,576	+0,0%	26,78	$8,475
01_farelo_de_soja	A_farelo_de_algodao (#60, $0,30)	Linear Nativo	Sucesso	$3,604	-1,8%	26,78	$8,449
01_farelo_de_soja	A_farelo_de_algodao (#60, $0,30)	Linear HiGHS	Sucesso	$3,606	-1,7%	26,78	$8,447
01_farelo_de_soja	A_farelo_de_algodao (#60, $0,30)	Não-Linear SLSQP	Sucesso	$3,613	+1,0%	26,78	$8,440
01_farelo_de_soja	B_farelo_de_canola (#31, $0,32)	Linear Nativo	Sucesso	$3,663	-0,2%	26,78	$8,390
01_farelo_de_soja	B_farelo_de_canola (#31, $0,32)	Linear HiGHS	Sucesso	$3,663	-0,2%	26,78	$8,390
01_farelo_de_soja	B_farelo_de_canola (#31, $0,32)	Não-Linear SLSQP	Sucesso	$3,680	+2,9%	26,78	$8,373
02_precos_dos_insumos	A_soja_menos30 (Soja -30%)	Linear Nativo	Sucesso	$3,132	-14,7%	26,78	$8,919
02_precos_dos_insumos	A_soja_menos30 (Soja -30%)	Linear HiGHS	Sucesso	$3,132	-14,7%	26,78	$8,919
02_precos_dos_insumos	A_soja_menos30 (Soja -30%)	Não-Linear SLSQP	Sucesso	$3,089	-13,6%	26,78	$8,962
02_precos_dos_insumos	B_soja_e_milho_mais30 (+30%)	Linear Nativo	Sucesso	$4,477	+22,0%	26,78	$7,574
02_precos_dos_insumos	B_soja_e_milho_mais30 (+30%)	Linear HiGHS	Sucesso	$4,477	+22,0%	26,78	$7,574
02_precos_dos_insumos	B_soja_e_milho_mais30 (+30%)	Não-Linear SLSQP	Sucesso	$4,358	+21,9%	26,78	$7,693
03_tolerancia_de_inclusao	A_largo_volumoso_estreito_mineral	Linear Nativo	Sucesso	$3,670	+0,0%	26,78	$8,381
03_tolerancia_de_inclusao	A_largo_volumoso_estreito_mineral	Linear HiGHS	Sucesso	$3,670	+0,0%	26,78	$8,381
03_tolerancia_de_inclusao	A_largo_volumoso_estreito_mineral	Não-Linear SLSQP	Sucesso	$3,576	+0,0%	26,78	$8,475
03_tolerancia_de_inclusao	B_estreito_todos (Buffer 5%)	Linear Nativo	Sucesso	$3,670	+0,0%	26,78	$8,381
03_tolerancia_de_inclusao	B_estreito_todos (Buffer 5%)	Linear HiGHS	Sucesso	$3,670	+0,0%	26,78	$8,381
03_tolerancia_de_inclusao	B_estreito_todos (Buffer 5%)	Não-Linear SLSQP	Sucesso	$3,576	+0,0%	26,78	$8,475
04_intervalo_de_reformulacao	A_30_dias	Linear Nativo	Sucesso	$3,639	-0,8%	27,03	$8,524
04_intervalo_de_reformulacao	A_30_dias	Linear HiGHS	Sucesso	$3,670	+0,0%	27,03	$8,493
04_intervalo_de_reformulac	A_30_dias	Não-	Sucess	$3,572	-0,1%	27,03	$8,591

ao		Linear SLSQP	o				
04_intervalo_de_reformulacao	B_14_dias	Linear Nativo	Sucesso	$3,687	+0,5%	27,33	$8,611
04_intervalo_de_reformulacao	B_14_dias	Linear HiGHS	Sucesso	$3,687	+0,5%	27,33	$8,611
04_intervalo_de_reformulacao	B_14_dias	Não-Linear SLSQP	Sucesso	$3,586	+0,3%	27,33	$8,712
05_tamanho_do_rebanho	A_100_vacas (Alta produção)	Linear Nativo	Sucesso	$3,598	-2,0%	47,40	$17,734
05_tamanho_do_rebanho	A_100_vacas (Alta produção)	Linear HiGHS	Sucesso	$3,598	-2,0%	47,40	$17,734
05_tamanho_do_rebanho	A_100_vacas (Alta produção)	Não-Linear SLSQP	Sucesso	$3,473	-2,9%	47,40	$17,859
06_buffer_de_fosforo	A_buffer_25 (Buffer 25%)	Linear Nativo	Sucesso	$3,670	+0,0%	26,78	$8,381
06_buffer_de_fosforo	A_buffer_25 (Buffer 25%)	Linear HiGHS	Sucesso	$3,670	+0,0%	26,78	$8,381
06_buffer_de_fosforo	A_buffer_25 (Buffer 25%)	Não-Linear SLSQP	Sucesso	$3,576	+0,0%	26,78	$8,475
06_buffer_de_fosforo	B_buffer_0 (Buffer 0%)	Linear Nativo	Sucesso	$3,670	+0,0%	26,78	$8,381
06_buffer_de_fosforo	B_buffer_0 (Buffer 0%)	Linear HiGHS	Sucesso	$3,670	+0,0%	26,78	$8,381
06_buffer_de_fosforo	B_buffer_0 (Buffer 0%)	Não-Linear SLSQP	Sucesso	$3,576	+0,0%	26,78	$8,475
08_aditivo_de_metano	A_sem_aditivo (Controle)	Linear Nativo	Sucesso	$3,670	+0,0%	26,78	$8,381
08_aditivo_de_metano	A_sem_aditivo (Controle)	Linear HiGHS	Sucesso	$3,670	+0,0%	26,78	$8,381
08_aditivo_de_metano	A_sem_aditivo (Controle)	Não-Linear SLSQP	Sucesso	$3,576	+0,0%	26,78	$8,475
08_aditivo_de_metano	B_monensina_24 (Monensina 24 mg)	Linear Nativo	Sucesso	$3,670	+0,0%	26,78	$8,381
08_aditivo_de_metano	B_monensina_24 (Monensina 24 mg)	Linear HiGHS	Sucesso	$3,670	+0,0%	26,78	$8,381
08_aditivo_de_metano	B_monensina_24 (Monensina 24 mg)	Não-Linear SLSQP	Sucesso	$3,576	+0,0%	26,78	$8,475
09_padrao_nutricional	A_NRC_2001 (Padrão NRC)	Linear Nativo	Sucesso	$3,679	+0,2%	26,78	$8,372
09_padrao_nutricional	A_NRC_2001 (Padrão NRC)	Linear HiGHS	Sucesso	$3,680	+0,3%	26,78	$8,371
09_padrao_nutricional	A_NRC_2001 (Padrão NRC)	Não-Linear SLSQP	Sucesso	$3,775	+5,6%	26,78	$8,276

Tabela 4. Matriz consolidada dos 10 experimentos de sensibilidade.

Análise do Experimento 07: Sensibilidade ao Preço de Venda do Leite

O preço do leite não altera a formulação da ração de mínimo custo, mas atua como determinante primário da margem sobre o custo alimentar (**IOFC**), conforme a Tabela 5.

Preço do Leite ($/kg)	Receita Bruta ($/vaca/dia)	Custo da Ração ($/vaca/dia)	IOFC ($/vaca/dia)	Variação do IOFC vs. Base ($0,45)
$0,35	$9,373	$3,576	$5,797	-31,6% (Cenário Crítico de Mercado)
$0,45 (Referência)	$12,051	$3,576	$8,475	Referência Padrão
$0,55	$14,729	$3,576	$11,154	+31,6% (Alta

				Rentabilidade)
$0,65	$17,408	$3,576	$13,832	+63,2% (Bonificação Extraordinária)

Tabela 5. Sensibilidade do IOFC ao preço de venda do leite.

6.3 Discussão e Interpretação dos Principais Achados

- **Alta elasticidade frente aos preços de soja e milho (02_precos_dos_insumos):** o experimento 02 confirmou a vulnerabilidade econômica da atividade leiteira diante da volatilidade do mercado de grãos: uma alta combinada de 30% em milho e farelo de soja eleva o custo diário em 22,0% (de $3,670 para $4,477 no Linear Nativo), reduzindo o IOFC em cerca de 9,6% (de $8,381 para $7,574). Em sentido oposto, uma queda de 30% no preço da soja reduz o custo em 14,7%.
- **Substituição proteica (01_farelo_de_soja):** a inclusão do Farelo de Algodão (#60, $0,30) representou economia de 1,8% na Programação Linear, mantendo o suprimento proteico e a produção de leite estáveis em 26,78 kg/dia.
- **Dinâmica do intervalo de reformulação (04_intervalo_de_reformulacao):** reduzir o intervalo de reotimização de 45 para 14 dias conferiu maior aderência à evolução biológica do rebanho, elevando a produção média de leite de 26,78 para 27,33 kg/dia e melhorando o IOFC real para $8,611 (Linear Nativo) e $8,712 (SLSQP).
- **Escala e diluição de custos (05_tamanho_do_rebanho):** no lote de 100 vacas com foco em alta produção (47,40 kg de leite/dia), a margem líquida IOFC mais do que dobrou (de $8,381 para $17,734 no Linear Nativo), evidenciando que a eficiência de conversão alimentar é o fator mais determinante para a lucratividade rural.
- **Independência da dieta em relação ao preço do leite e aos aditivos de metano:** a suíte comprovou empiricamente que o preço do leite (experimento 07) e aditivos como a Monensina (experimento 08, nas doses testadas) não deslocam o ponto de ótimo de mínimo custo na formulação linear, preservando a coerência conceitual do modelo. O mesmo ocorreu com as tolerâncias de inclusão (experimento 03) e com o buffer de fósforo (experimento 06).
- **Padrão nutricional (09_padrao_nutricional):** sob o padrão NRC (2001), os solvers lineares mantiveram custos praticamente iguais aos do baseline ($3,679 e $3,680), enquanto o SLSQP apresentou custo 5,6% maior ($3,775).
- **Convergência e estabilidade numérica:** em todas as variantes testadas, os solvers Linear Nativo (Simplex Duas Fases) e Linear HiGHS apresentaram soluções numericamente equivalentes, e o solver Não-Linear SLSQP convergiu com sucesso em todas as variantes, confirmando a confiabilidade do motor biofísico após a inclusão das fontes minerais concentradas.

7. Conclusões e Recomendações

1. **Três Solvers Plenamente Operacionais e Auditáveis:** O sistema RuFaS agora disponibiliza uma suíte completa de otimização:
   - **HiGHS Simplex:** Para produção em escala e simulações rápidas.
   - **Pure Simplex Nativo:** Para ensino de Pesquisa Operacional, validação acadêmica e auditoria passo a passo via tableaus interativos.
   - **SLSQP:** Para modelagem biológica não-linear e respostas fisiológicas acopladas.
2. **Viabilidade dos Solvers Garantida:** A inclusão das fontes minerais concentradas (#230 Carbonato de Cálcio, #231 Fosfato Bicálcico e #239 Sal Comum) eliminou em

100% o conflito matemático entre suprimento de minerais e teto de proteína metabolizável, tornando ambos os modelos (NASEM e NRC) plenamente operacionais.

3. **Auditoria e Segurança no Suporte à Decisão:** O mecanismo de checagem implementado (`check_formulation_fallback`) garante que falhas de formulação não passem despercebidas, protegendo consultores, zootecnistas e produtores rurais de tomar decisões baseadas em contingências ocultas do simulador.
4. **Recomendação de Uso Prático:**
   - Utilizar a aba **“ Tableaus Simplex”** para aulas, relatórios acadêmicos e compreensão exata das restrições nutricionais ativas (custos reduzidos e preços-sombra).
   - Utilizar o **Solver Linear (HiGHS Simplex)** para triagens econômicas anuais rápidas e sensibilidade de mercado.
   - Utilizar o **Solver Não-Linear (SLSQP)** com **NASEM** para estudos zootécnicos finos de emissão entérica e máxima produtividade.

---

8. Guia Prático de Instalação, Configuração e Execução

Esta seção fornece o passo a passo completo e reprodutível para configurar o ambiente de desenvolvimento, instalar as dependências e executar o sistema RuFaS tanto em ambiente Windows quanto Linux/macOS.

8.1 Pré-requisitos de Sistema

Antes de iniciar, certifique-se de possuir instalado em sua máquina: * **Python:** Versão **3.12** ou **3.13** (conforme especificado no arquivo `pyproject.toml`: `>=3.12, <3.14`). * **Git:** Para clonagem e versionamento do código. * **Navegador Web Moderno:** Google Chrome, Mozilla Firefox, Microsoft Edge ou Safari.

```
# Verificar a versão instalada do Python
python --version   # ou python3 --version
```

8.2 Clonagem e Criação do Ambiente Virtual (venv)

Recomenda-se fortemente a utilização de um ambiente virtual isolado para evitar conflitos de bibliotecas com o Python global do sistema.

No Windows (PowerShell ou Prompt de Comando):

```
# 1. Navegue até o diretório do projeto
cd RuFaS – Pesquisa Operacional

# 2. Crie o ambiente virtual (caso ainda não exista)
python -m venv venv

# 3. Ative o ambiente virtual
# No PowerShell:
.\venv\Scripts\Activate.ps1
# No Prompt de Comando (CMD):
.\venv\Scripts\activate.bat
```

No Linux / macOS (Bash / Zsh):

```
# 1. Navegue até a raiz do repositório
cd RuFaS-dev/RuFaS-dev

# 2. Crie o ambiente virtual
python3 -m venv venv

# 3. Ative o ambiente virtual
source venv/bin/activate
```

8.3 Instalação das Dependências

Com o ambiente virtual ativado, instale o motor biofísico do RuFaS em modo editável (`-e`), o framework web Flask e os pacotes científicos de Pesquisa Operacional:

```
# Atualize o gerenciador de pacotes pip
python -m pip install --upgrade pip

# Instale o pacote RuFaS em modo de desenvolvimento com as dependências do pyproject.toml
pip install -e .

# Instale as dependências da aplicação Web (Flask)
pip install Flask
```

Pacotes Principais Instalados:

* `numpy==2.2.0` (Álgebra linear do Pure Simplex Solver)
* `scipy>=1.15.1` (Solvers HiGHS Simplex e SLSQP)
* `pandas>=2.2.3` (Processamento de séries temporais das simulações)
* `matplotlib>=3.10.0` (Renderização de gráficos biofísicos)
* `Flask>=3.1.0` (Servidor de APIs e Dashboard Web)
* `numba>=0.61.0` (Aceleração JIT do modelo animal)

8.4 Modos de Execução da Aplicação

O ecossistema dispõe de **três modos de execução**, adaptados para diferentes perfis de usuário:

Modo 1: Interface Gráfica Web Interativa (Recomendado)

Inicia o servidor Flask local e abre automaticamente o painel completo no seu navegador padrão.

- **No Windows:** Dê dois cliques no arquivo `iniciar_painel.bat` ou execute no terminal:

  ```
  iniciar_painel.bat
  ```

- **No Linux / macOS:** Dê permissão de execução e dispare o script `iniciar_painel.sh`:

```
chmod +x iniciar_painel.sh
./iniciar_painel.sh
```

- **Execução Direta via Python (Multiplataforma):**

```
python web_app/app.py
```

Após iniciar, o painel estará disponível no endereço: 👉 **http://127.0.0.1:5000**

Na interface web, você poderá:

1. Alternar entre os padrões **NASEM** e **NRC** na aba *"Cenários & Fazendas"*.

2. Selecionar entre os solvers **Linear (HiGHS)**, **Linear Nativo (Simplex Duas Fases)** ou **Não-Linear (SLSQP)**.

3. Disparar simulações com barra de progresso em tempo real na aba *"Painel Geral"*.

4. Inspecionar cada passo matricial na aba interativa *" 🎲 Tableaus Simplex"*.

5. Visualizar a ficha de desempenho individual e zootécnico de cada vaca do rebanho na aba *"Ficha das Vacas"*.

Modo 2: Execução em Linha de Comando (CLI / Headless)

Para rodar simulações em servidores remotos, pipelines CI/CD ou testes automatizados em lote sem interface gráfica:

```
# Executa a simulação padrão com rastreamento e arquivamento automático
python main.py
```

Você pode especificar o solver desejado diretamente definindo a variável de ambiente `RUFAS_OPTIMIZATION_MODE`:

- **Linear Nativo Autônomo (Simplex em Duas Fases com gravação de Tableaus):**

```
# Windows PowerShell:
$env:RUFAS_OPTIMIZATION_MODE = "linear_native"; python main.py
# Linux / macOS:
RUFAS_OPTIMIZATION_MODE=linear_native python main.py
```

- **Linear HiGHS (Biblioteca C++):**

```
# Windows PowerShell:
$env:RUFAS_OPTIMIZATION_MODE = "linear"; python main.py
# Linux / macOS:
RUFAS_OPTIMIZATION_MODE=linear python main.py
```

- **Não-Linear Mecanístico (SLSQP):**

```
# Windows PowerShell:
$env:RUFAS_OPTIMIZATION_MODE = "nonlinear"; python main.py
# Linux / macOS:
RUFAS_OPTIMIZATION_MODE=nonlinear python main.py
```

Ao término, a simulação será automaticamente arquivada na pasta `output/simulacoes/<FAZENDA>_SIM<N>_<TIMESTAMP>/` contendo os parâmetros, logs de console, arquivos CSV brutos e o manifesto JSON auditável.

Modo 3: Scripts de Verificação e Linting

Para verificar a integridade sintática e tipagem estática do código modificado com Flake8 e MyPy:

- **No Windows:**

  ```
  check_changes.bat main
  ```

- **No Linux / macOS:**

  ```
  chmod +x check_changes.sh
  ./check_changes.sh main
  ```

8.5 Solução de Problemas Comuns (Troubleshooting)

Sintoma / Erro	Causa Provável	Ação Corretiva
`Address already in use` (Porta 5000 ocupada)	Uma instância anterior do Flask continua rodando em segundo plano.	No Windows, execute `Get-NetTCPConnection -LocalPort 5000` e finalize o PID via `Stop-Process -Id <PID>`. No Linux, execute `fuser -k 5000/tcp`.
`ModuleNotFoundError: No module named 'RUFAS'`	O pacote RuFaS não foi instalado em modo editável no ambiente virtual ativo.	Com o venv ativado, execute `pip install -e .` a partir da raiz do repositório.
`Execution of scripts is disabled on this system` (PowerShell)	Política de restrição de execução de scripts padrão do Windows.	Execute no PowerShell como Administrador: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.
`Permission denied: ./iniciar_painel.sh`	Falta de permissão de execução no script bash (Linux/Mac).	Execute `chmod +x iniciar_painel.sh check_changes.sh`.

Tabela 6. Soluções de Problemas.

Relatório técnico consolidado no ambiente RuFaS-PesquisaOperacional. Todos os códigos, modelos matemáticos e artefatos de dados estão disponíveis no repositório local.

Referências Bibliográficas

RUMINANT FARM SYSTEMS. **RuFaS — Ruminant Farm Systems: repositório oficial do projeto**. GitHub. Disponível em: https://github.com/RuminantFarmSystems/RuFaS. Acesso em: 24 out. 2026.

ROCHA JÚNIOR, Ivânio José da; SILVA, Robson Pereira da. **RuFaS-PesquisaOperacional: implementação e análise de solvers de otimização**. GitHub, 2026. Disponível em: https://github.com/ivaniojr/RuFaS-PesquisaOperacional.git. Acesso em: 24 out. 2026.

FREITAS, Leandro Alexandre. **RuFaS: a fazenda leiteira como problema de otimização — e como o RuFaS a resolve**. Material didático da Pós-Graduação em Inteligência Artificial. Instituto Federal de Goiás (IFG), [s.d.]. 13 páginas. Documento fornecido pelo docente.

RUMINANT FARM SYSTEMS. **Documentação técnica do RuFaS**. Disponível em: https://ruminantfarmsystems.github.io/RuFaS/. Acesso em: 24 out. 2026.